



SISTEMAS MULTIAGENTE E APRENDIZADO POR REFORÇO
APLICADOS A META-HEURÍSTICAS PARA OTIMIZAÇÃO DO
PROBLEMA DE ROTEAMENTO DE VEÍCULOS

Hugo de Barros Araujo

Projeto de Graduação apresentado ao Curso
de Engenharia Eletrônica e de Computação
da Escola Politécnica, Universidade Federal
do Rio de Janeiro, como parte dos requisitos
necessários à obtenção do título de Engenheiro.

Orientador: Pedro Henrique Gonzalez Silva

Rio de Janeiro
Fevereiro de 2026

SISTEMAS MULTIAGENTE E APRENDIZADO POR REFORÇO
APLICADOS A META-HEURÍSTICAS PARA OTIMIZAÇÃO DO
PROBLEMA DE ROTEAMENTO DE VEÍCULOS

Hugo de Barros Araujo

PROJETO DE GRADUAÇÃO SUBMETIDO AO CORPO DOCENTE DO CURSO
DE ENGENHARIA ELETRÔNICA E DE COMPUTAÇÃO DA ESCOLA PO-
LITÉCNICA DA UNIVERSIDADE FEDERAL DO RIO DE JANEIRO COMO
PARTE DOS REQUISITOS NECESSÁRIOS PARA A OBTENÇÃO DO GRAU
DE ENGENHEIRO ELETRÔNICO E DE COMPUTAÇÃO

Autor:

Hugo de Barros Araujo

Orientador:

Pedro Henrique Gonzalez Silva, Ph. D.

Examinador:

Prof Frances Elizabeth Allen, D. Sc.

Examinador:

Prof. Alan Jay Perlis, D. E.

Rio de Janeiro
Fevereiro de 2026

Declaração de Autoria e de Direitos

Eu, *Hugo de Barros Araujo*, autor da monografia *SISTEMAS MULTIA-GENTE E APRENDIZADO POR REFORÇO APLICADOS A META-HEURÍSTICAS PARA OTIMIZAÇÃO DO PROBLEMA DE ROTEAMENTO DE VEÍCULOS*, subscrevo para os devidos fins, as seguintes informações:

1. O autor declara que o trabalho apresentado na disciplina de Projeto de Graduação da Escola Politécnica da UFRJ é de sua autoria, sendo original em forma e conteúdo.
2. Excetua-se do item 1. eventuais transcrições de texto, figuras, tabelas, conceitos e idéias, que identifiquem claramente a fonte original, explicitando as autorizações obtidas dos respectivos proprietários, quando necessárias.
3. O autor permite que a UFRJ, por um prazo indeterminado, efetue em qualquer mídia de divulgação, a publicação do trabalho acadêmico em sua totalidade, ou em parte. Essa autorização não envolve ônus de qualquer natureza à UFRJ, ou aos seus representantes.
4. O autor pode, excepcionalmente, encaminhar à Comissão de Projeto de Graduação, a não divulgação do material, por um prazo máximo de 01 (um) ano, improrrogável, a contar da data de defesa, desde que o pedido seja justificado, e solicitado antecipadamente, por escrito, à Congregação da Escola Politécnica.
5. O autor declara, ainda, ter a capacidade jurídica para a prática do presente ato, assim como ter conhecimento do teor da presente Declaração, estando ciente das sanções e punições legais, no que tange a cópia parcial, ou total, de obra intelectual, o que se configura como violação do direito autoral previsto no Código Penal Brasileiro no art.184 e art.299, bem como na Lei 9.610.
6. O autor é o único responsável pelo conteúdo apresentado nos trabalhos acadêmicos publicados, não cabendo à UFRJ, aos seus representantes, ou ao(s) orientador(es), qualquer responsabilização/ indenização nesse sentido.
7. Por ser verdade, firmo a presente declaração.

Hugo de Barros Araujo

UNIVERSIDADE FEDERAL DO RIO DE JANEIRO

Escola Politécnica - Departamento de Eletrônica e de Computação

Centro de Tecnologia, bloco H, sala H-217, Cidade Universitária

Rio de Janeiro - RJ CEP 21949-900

Este exemplar é de propriedade da Universidade Federal do Rio de Janeiro, que poderá incluí-lo em base de dados, armazenar em computador, microfilmear ou adotar qualquer forma de arquivamento.

É permitida a menção, reprodução parcial ou integral e a transmissão entre bibliotecas deste trabalho, sem modificação de seu texto, em qualquer meio que esteja ou venha a ser fixado, para pesquisa acadêmica, comentários e citações, desde que sem finalidade comercial e que seja feita a referência bibliográfica completa.

Os conceitos expressos neste trabalho são de responsabilidade do(s) autor(es).

AGRADECIMENTO

Ao povo brasileiro, que financia esta universidade e tornou possível minha formação em uma instituição de excelência como a UFRJ. Que a educação pública, gratuita e de qualidade continue sendo um pilar de transformação, tornando-se cada vez mais inclusiva e acessível a todos.

À minha família, a base de tudo o que sou. Um agradecimento profundo e especial aos meus pais, José Antonio e Rosália, que não mediram esforços para que eu chegasse até aqui. O apoio incondicional, os sacrifícios silenciosos e o incentivo constante foram o combustível necessário para concluir esta jornada. Vocês são meu maior exemplo de dedicação e amor.

Ao meu tio e padrinho, Lorrán, quem mais me ajudou a despertar o gosto pela ciência.

À minha namorada, Eduarda, parceira de todas as horas. Obrigado por compartilhar a vida comigo, por deixar tudo sempre leve e alegre quando estou com você.

RESUMO

Este trabalho aborda a resolução do Problema de Roteamento de Veículos com Janelas de Tempo (*Vehicle Routing Problem with Time Windows* - VRPTW), um desafio de otimização combinatória classificado como *NP-hard*, cujas aplicações práticas são fundamentais para a eficiência logística e redução de custos operacionais. O objetivo central é o desenvolvimento de um sistema híbrido que integra meta-heurísticas clássicas a um Sistema Multiagente (*Multi-Agent System* - MAS) orquestrado por Aprendizado por Reforço (*Reinforcement Learning* - RL). Inicialmente, foram implementados de forma independente os algoritmos de *Simulated Annealing*, *Tabu Search*, e *Genetic Algorithm*, utilizando as instâncias de Solomon como *benchmark* para avaliação de desempenho.

A originalidade da proposta reside na encapsulação desses algoritmos como agentes especialistas dentro do *framework* Mesa, onde a cooperação é estabelecida através de uma base de soluções de elite e da estratégia de *Path Relinking*, que explora trajetórias entre soluções de alta qualidade para evitar ótimos locais. Para elevar a eficiência do sistema, aplicou-se o algoritmo *Q-Learning* como uma hiper-heurística capaz de aprender, de forma autônoma, qual especialista deve ser acionado em diferentes estados do espaço de busca. Os resultados sugerem que a inteligência coletiva e a adaptação dinâmica de parâmetros via RL superam o desempenho individual dos algoritmos, proporcionando soluções otimizadas em tempo computacional viável.

Palavras-chave: Otimização Combinatória, VRPTW, Sistemas Multiagentes, Meta-heurísticas, Aprendizado por Reforço, *Path Relinking*.

ABSTRACT

This work addresses the resolution of the Vehicle Routing Problem with Time Windows (VRPTW), a NP-hard combinatorial optimization challenge with critical applications in logistics efficiency and operational cost reduction. The primary objective is to develop a hybrid system that integrates classical metaheuristics within a Multi-Agent System (MAS) orchestrated by Reinforcement Learning (RL). Initially, Simulated Annealing, Tabu Search, and Genetic Algorithms were independently implemented and evaluated using Solomon’s benchmark instances to establish performance baselines.

The originality of this approach lies in encapsulating these algorithms as specialized agents within the Mesa framework, where cooperation is established through a shared elite solution pool and the Path Relinking strategy, which explores trajectories between high-quality solutions to circumvent local optima. To enhance system efficiency, the Q-Learning algorithm was applied as a hyper-heuristic capable of autonomously learning which specialist to activate across different states of the search space. Preliminary results suggest that collective intelligence and dynamic parameter adaptation via RL outperform individual algorithmic performance, providing optimized solutions within viable computational time.

Key-words: Combinatorial Optimization, VRPTW, Multi-Agent Systems, Metaheuristics, Reinforcement Learning, Path Relinking.

SIGLAS

GA – *Genetic Algorithm* (algoritmo genético)

I/O – *In/Out* (entrada/saída)

MAS – *Multi-Agent System* (sistema multiagente)

MAS-RL – *Multi-Agent System with Reinforcement Learning* (sistema multiagente com aprendizado por reforço)

NP-hard – *Non-deterministic Polynomial-time hard* (problemas de tempo polinomial não-determinístico difícil)

OOP – *Object-Oriented Programming* (programação orientada a objetos)

PR – *Path Relinking* (reconexão por caminhos)

RAM – *Random Access Memory* (memória de acesso aleatório)

RL – *Reinforcement Learning* (aprendizado por reforço)

SA – *Simulated Annealing* (recozimento simulado)

TS – *Tabu Search* (busca tabu)

TSP – *Traveling Salesman Problem* (problema do caixeiro viajante)

UFRJ – Universidade Federal do Rio de Janeiro

VRP – *Vehicle Routing Problem* (problema de roteamento de veículos)

VRPTW – *Vehicle Routing Problem with Time Windows* (problema de roteamento de veículos com janelas de tempo)

Sumário

1	Introdução	1
1.1	Tema	1
1.2	Delimitação	2
1.3	Justificativa	3
1.4	Objetivos	3
1.5	Metodologia	4
1.6	Descrição	5
2	Fundamentação Teórica	6
2.1	Definição do problema	6
2.1.1	O Problema de Roteamento de Veículos	7
2.1.2	Metodologia de avaliação e o Benchmark de Solomon	7
2.2	Meta-heurísticas selecionadas	9
2.2.1	Recozimento Simulado	9
2.2.2	Busca Tabu	10
2.2.3	Algoritmo Genético	10
2.3	Sistema multiagente	11
2.3.1	Framework Mesa	11
2.3.2	Estratégia de cooperação: <i>Path Relinking</i>	12
2.4	Aprendizado por Reforço	15
2.5	Avaliação estatística de desempenho algorítmico	18
2.5.1	Teste de Friedman	18
2.5.2	Teste Post-Hoc de Nemenyi	19
3	Método Proposto	21

3.1	Etapas iniciais	21
3.2	Implementação de sistema multiagente	23
3.2.1	Arquitetura e componentes do sistema	24
3.2.2	Cooperação e a estratégia de Path Relinking	24
3.3	Hiper-heurística baseada em Aprendizado por Reforço	26
4	Experimentos Computacionais	31
4.1	Software e hardware	31
4.2	Instâncias de teste	32
4.3	Detalhes de implementação e parametrização	32
4.4	Resultados dos experimentos	34
4.5	Análise estatística	40
5	Conclusão	44
5.1	Resumo dos resultados finais	44
5.2	Oportunidades de melhoria	45
	Bibliografia	47
A	Benchmark de Solomon	49
B	Estimativa para busca exaustiva	52
C	Resultados dos experimentos	54

Lista de Figuras

2.1	Diagrama de Path Relinking	14
2.2	Visão geral do Q-Learning	16
2.3	Algoritmo básico de Q-Learning	17
3.1	Distribuição da instância RC201	22
3.2	Rotas ótimas da instância RC201	23
3.3	Fluxograma do MAS	25
3.4	Fluxograma do MAS-RL	29
4.1	Distribuição do gap de frota	35
4.2	Distribuição do gap de distância	35
4.3	Evolução temporal do gap de frota	37
4.4	Evolução temporal do gap de distância	38
4.5	Análise de convergência por instância	39
4.6	Diagrama de Diferença Crítica (Nemenyi) para a minimização de Veículos.	42
5.1	Tempo de viagem, espera e serviço	46
A.1	Mapa da instância C101	49
A.2	Mapa da instância R101	50
A.3	Mapa da instância RC101	50

Lista de Tabelas

4.1	Parâmetros de implementação dos experimentos.	33
4.2	Matriz de Resultados Agregados	41
4.3	Matriz de p-valores do Teste Post-Hoc de Nemenyi e ranks médios	41
C.1	Resultados completos por instância e método.	54

Capítulo 1

Introdução

Este capítulo introduz o leitor ao contexto no qual a presente pesquisa se insere, apresentando as motivações iniciais, o escopo do problema de otimização selecionado e a relevância científica da abordagem proposta. A Seção 1.1 delinea o tema central do estudo, abordando as restrições e a hierarquia do Problema de Roteamento de Veículos com Janelas de Tempo (VRPTW). Em seguida, a Seção 1.2 estabelece as delimitações do trabalho, focando no desenvolvimento e avaliação empírica de meta-heurísticas integradas em um sistema multiagente colaborativo guiado por aprendizado por reforço. A justificativa para a aplicação destas ferramentas algorítmicas, dadas as dificuldades inerentes aos métodos exatos diante de problemas classificados como *NP-hard*, é apresentada na Seção 1.3. Por fim, as Seções 1.4, 1.5 e 1.6 expõem, respectivamente, os objetivos formulados, a metodologia adotada de melhoria contínua e a estruturação capitular do conteúdo subsequente neste documento.

1.1 Tema

O tema do trabalho é a aplicação e a análise de métodos meta-heurísticos para a resolução de problemas de otimização complexos, classificados como *NP-hard* [1], como o Problema de Roteamento de Veículos (ou *Vehicle Routing Problem* – VRP). O VRP, introduzido pioneiramente na literatura sob a ótica da distribuição de derivados de petróleo em 1959 [2], consiste em determinar o conjunto ideal de rotas para uma frota de veículos, com o objetivo de visitar um conjunto de clientes, cada

um com uma demanda específica. O problema a ser resolvido é encontrar a solução mais eficiente para este roteamento, considerando restrições como distância total, número de veículos, tempo e capacidade de entrega.

Dentro do vasto domínio do VRP, este trabalho concentra-se especificamente no Problema de Roteamento de Veículos com Janelas de Tempo (*Vehicle Routing Problem with Time Windows* – VRPTW). O VRPTW estende o problema clássico ao impor que o atendimento a cada cliente deve ser iniciado obrigatoriamente dentro de um intervalo temporal pré-definido, composto por um tempo de abertura e um tempo de fechamento (*time window*).

A inclusão destas restrições temporais torna o desafio ainda mais rigoroso, pois a viabilidade de uma rota não depende apenas da capacidade de carga do veículo, mas da coordenação precisa dos horários de chegada e dos tempos de serviço em cada nó. Neste cenário, o objetivo de encontrar a solução mais eficiente desdobra-se em uma hierarquia de otimização: busca-se, primordialmente, a minimização do número de veículos necessários e, subsequentemente, a redução da distância total percorrida, garantindo que nenhum compromisso temporal seja violado. Por ser um problema de natureza *NP-hard*, a exploração de janelas de tempo exige que os métodos meta-heurísticos aplicados possuam alta capacidade de adaptação para navegar em um espaço de busca repleto de restrições de inviabilidade.

1.2 Delimitação

O objeto de estudo são problemas de otimização *NP-hard*. O projeto se concentrará na otimização do VRPTW, que estende o clássico Problema do Caixeiro Viajante (*Traveling Salesman Problem* – TSP) para múltiplos veículos, capacidade de carga e janelas de tempo. O foco será na implementação e comparação de diferentes algoritmos meta-heurísticos (*Simulated Annealing* – SA, *Tabu Search* – TS e *Genetic Algorithm* – GA), e na integração desses métodos com um sistema multiagente (*Multi-Agent System* – MAS), conjuntamente com a aplicação de aprendizado por reforço (*Reinforcement Learning* – RL) para melhoria deste sistema.

1.3 Justificativa

O VRPTW, em virtude de sua formulação que estende o VRP clássico, é rigorosamente classificado como um problema *NP-hard* [1]. Nesse contexto de alta complexidade computacional, o esforço temporal demandado por métodos exatos para a obtenção de uma solução ótima cresce de forma exponencial em função da dimensionalidade da instância analisada, tornando tais abordagens intratáveis para a ampla escala de cenários reais. Diante dessa limitação, as meta-heurísticas despontam como uma alternativa viável, visto que, embora não assegurem a otimalidade global, oferecem mecanismos para explorar o espaço de busca eficientemente, fornecendo aproximações de alta qualidade em tempo computacional factível.

A originalidade deste trabalho reside na aplicação de um MAS com RL para integrar e aprimorar o desempenho dos diferentes métodos meta-heurísticos. O objetivo é explorar como a comunicação entre agentes pode levar a resultados superiores aos de cada algoritmo individualmente, evitando mínimos locais e alcançando uma solução globalmente mais eficiente. A importância do projeto está em sua aplicabilidade prática em áreas como logística, transporte e serviços de entrega, onde a otimização de rotas é fundamental para a eficiência e redução de custos.

Ainda no tocante à justificativa da abordagem, o Apêndice B apresenta uma análise quantitativa da grandeza temporal requerida pela força bruta para a obtenção da solução ótima nas instâncias discutidas, reiterando a utilidade de métodos heurísticos.

1.4 Objetivos

O objetivo geral é analisar e implementar um sistema híbrido, combinando métodos meta-heurísticos com um sistema multiagente e aprendizado por reforço, para otimizar a resolução do VRPTW. Como objetivos específicos, temos:

1. Formular o VRPTW, definindo a função de custo e as restrições (tempo, volume, peso) para a avaliação das soluções;

2. Implementar, de forma independente, os algoritmos meta-heurísticos: SA, TS e GA;
3. Integrar os algoritmos em um MAS, no qual a interação entre os agentes leva à colaboração na busca pela solução;
4. Aplicar RL para aprimorar a estratégia de busca dos agentes;
5. Comparar o desempenho dos algoritmos individuais com o MAS e o sistema com aprendizado por reforço (MAS-RL) para avaliar a melhoria na otimização da solução.

1.5 Metodologia

A metodologia será dividida nas seguintes etapas, focando na aplicação de uma abordagem iterativa e de melhoria contínua para a otimização do problema.

- **Modelagem do Problema:** O VRP será formulado como um problema de otimização, no qual uma solução é representada como um vetor ordenado de pontos de entrega. A função de custo avaliará a qualidade de cada solução, considerando a distância percorrida, o número de veículos e as restrições de tempo e capacidade, atribuindo pesos a cada parâmetro;
- **Implementação dos Algoritmos:** Os três algoritmos meta-heurísticos serão implementados em *Python*. Para cada algoritmo, serão definidas etapas de inicialização de uma solução aleatória, geração de novas soluções e a manutenção de um histórico para direcionar a busca;
- **Integração com Sistema Multiagente:** Os algoritmos serão encapsulados como agentes dentro de um MAS. O sistema operará de forma cooperativa, permitindo que os agentes interajam para compartilhar informações e refinar as soluções encontradas;
- **Aplicação de Aprendizado por Reforço:** O método de *Q-Learning* será aplicado para associar os estados do problema às ações (ajuste de estratégias de busca), permitindo que o sistema aprenda a selecionar a melhor abordagem de forma autônoma ao longo do tempo.

1.6 Descrição

O restante deste trabalho está organizado em quatro capítulos principais, além das referências bibliográficas e apêndices, conforme detalhado a seguir.

No Capítulo 2, é apresentada a Fundamentação Teórica, que estabelece a base conceitual necessária para a compreensão do projeto. Nele, define-se o VRPTW e detalha-se a metodologia de avaliação baseada no *Benchmark* de Solomon [3]. O capítulo descreve o funcionamento técnico do trio de meta-heurísticas selecionado e introduz a arquitetura do MAS utilizando o *framework* Mesa. Finaliza abordando as estratégias de cooperação via *Path Relinking* (PR) e a teoria de Aprendizado por Reforço.

O Capítulo 3 detalha o desenvolvimento do Método Proposto da pesquisa. Ele descreve as etapas iniciais de modelagem e as arquiteturas das classes. Em seguida, expõe-se a modelagem do sistema multiagente, demonstrando a dinâmica de cooperação por meio do *Path Relinking* e a implementação do *Q-Learning* como hiper-heurística, orquestrando e selecionando as melhores ações para esse ambiente.

Os testes realizados e a apresentação dos resultados ficam a encargo do Capítulo 4: Experimentos Computacionais. A seção avalia o desempenho prático, apresentando inicialmente o *baseline* de cada meta-heurística isolada, em sequência os resultados agregados pela cooperação simplista do sistema multiagente e, por fim, a análise do desempenho da hiper-heurística com Aprendizado por Reforço (MAS-RL).

As reflexões e fechamentos a respeito da pesquisa são apresentados no Capítulo 5, que compõe as Conclusões do trabalho. Nele, será explicitado se a integração entre MAS e RL de fato superou o desempenho dos algoritmos isolados, validando a hipótese de que a inteligência coletiva e a adaptação dinâmica de parâmetros produzam soluções superiores para o VRPTW. O capítulo encerra com a indicação de oportunidades de melhoria e sugestões para trabalhos futuros.

Capítulo 2

Fundamentação Teórica

Este capítulo apresenta a fundamentação teórica essencial para o entendimento das metodologias e soluções computacionais propostas neste trabalho. A Seção 2.1 formaliza o conceito do Problema de Roteamento de Veículos e detalha os padrões de avaliação de instâncias da literatura. A Seção 2.2 apresenta as meta-heurísticas adotadas (Recozimento Simulado, Busca Tabu e Algoritmo Genético) descrevendo sucintamente seus funcionamentos e premissas. Posteriormente, a Seção 2.3 explora os princípios de um Sistema multiagente, abordando o *framework* de modelagem distribuída escolhido e a estratégia de cooperação implementada, o *Path Relinking*. A compreensão do Aprendizado por Reforço e a lógica da recompensa empregada na estratégia do *Q-Learning*, que norteia a coordenação estratégica proposta, são objetos da Seção 2.4. Por fim, a Seção 2.5 discute a importância de uma avaliação estatística rigorosa para a comparação de algoritmos meta-heurísticos, destacando os testes de hipóteses adequados para validar as conclusões obtidas a partir dos experimentos computacionais realizados.

2.1 Definição do problema

Para compreender a avaliação das soluções quantitativas na experimentação futura, faz-se necessário contextualizar a natureza exata e as particularidades do problema central que pauta toda a sequência teórica e empírica deste estudo. Inicialmente, a Subseção 2.1.1 define o arranjo lógico do conhecido e amplamente mapeado VRP, englobando as dificuldades computacionais e as raízes atadas à complexidade

tempo de polinomial não-determinístico difícil (*NP-hard*). Secundariamente, a Subseção 2.1.2 elucida a base paramétrica de referências extraída do Benchmark de Solomon de 1987, estabelecendo um padrão ouro viável de contraste na apuração da acurácia e otimização obtida pelas composições desenvolvidas em tempo de execução comparado.

2.1.1 O Problema de Roteamento de Veículos

O Problema de Roteamento de Veículos (VRP) é um desafio de otimização combinatoria e programação linear inteira que busca determinar o conjunto ideal de rotas para uma frota de veículos entregarem mercadorias a um conjunto específico de clientes [2, 4]. De forma genérica, o objetivo fundamental do VRP é minimizar o custo total da operação (geralmente expresso em distância total ou tempo) garantindo que:

- Cada cliente deve ser visitado exatamente uma vez;
- Todos os veículos devem começar e terminar suas rotas em um depósito central;
- As restrições operacionais (como capacidade de carga ou janelas de tempo) devem ser estritamente respeitadas.

O VRP é classificado na teoria da complexidade computacional como um problema *NP-hard* [1]. Do ponto de vista teórico, isso implica que qualquer problema da classe NP pode ser reduzido ao VRP em tempo polinomial. Na prática, significa que não se conhece nenhum algoritmo capaz de encontrar a solução ótima para o problema em tempo polinomial. É precisamente essa complexidade computacional inerente que fundamenta a aplicação das meta-heurísticas (SA, TS e GA) investigadas neste estudo. Em contraste com os métodos de resolução exatos, as abordagens meta-heurísticas são concebidas para encontrar soluções subótimas de alta qualidade em tempo que viabilize a aplicação prática em cenários de envergadura real.

2.1.2 Metodologia de avaliação e o Benchmark de Solomon

Para assegurar a validade científica deste estudo e permitir a comparabilidade dos resultados obtidos pelas meta-heurísticas propostas, a modelagem será conduzida

estritamente sob os parâmetros do Benchmark de Solomon (1987). A utilização de instâncias consagradas na literatura é imperativa, pois fornecem um “gabarito” técnico que permite atestar a precisão e a eficiência de cada algoritmo desenvolvido.

Dado que as instâncias de Solomon incorporam coordenadas geográficas, demandas de carga e janelas temporais, o problema aqui tratado é classificado como Problema de Roteamento de Veículos com Janelas de Tempo (VRPTW). Embora simplificada em relação às complexidades imprevisíveis do mundo real (como variações de tráfego ou clima), essa modelagem oferece um ambiente controlado e puramente matemático para a análise de desempenho algorítmico. Para que a comparação seja viável e metodologicamente aceitável, a instanciação seguirá as métricas rigorosas do *benchmark*:

1. **Função Objetivo Hierárquica:** A avaliação do desempenho seguirá uma prioridade de dois níveis:
 - (a) **Minimização do número de veículos (K):** O objetivo primário é atender a todos os clientes com a menor frota possível.
 - (b) **Minimização da distância total percorrida (D):** Como objetivo secundário, busca-se o percurso mais curto para a frota definida.
2. **Cálculo de Distância e Tempo:** A distância entre dois pontos (i, j) é calculada pela Distância Euclidiana plana:
$$d_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$$
3. Conforme as regras de Solomon, o tempo de viagem é numericamente equivalente à distância ($t_{ij} = d_{ij}$), e todos os cálculos devem utilizar precisão dupla (*double precision*), com os resultados finais de distância arredondados para duas casas decimais.
4. **Ponto de Origem e Retorno:** O cliente indexado como 0 define o depósito central, ponto obrigatório de partida e encerramento de todas as rotas.
5. **Restrições de Viabilidade:** Uma solução para o problema é considerada válida apenas se garantir o atendimento da demanda de todos os clientes em sua

totalidade. Além disso, cada rota individual deve respeitar rigorosamente as seguintes restrições operacionais: a soma das demandas dos clientes inseridos em uma mesma rota não pode exceder a Capacidade Máxima do Veículo (Q); e o serviço deve ser iniciado e finalizado dentro da Janela de Tempo estabelecida pelo cliente (*Ready Time* e *Due Date*), computando-se obrigatoriamente o Tempo de Serviço (*Service Time*) em cada parada.

Esta abordagem padronizada transforma o problema em um estudo de caráter matemático e exploratório, estabelecendo uma base sólida que pode, futuramente, ser adaptada para modelos logísticos de maior complexidade e variáveis dinâmicas.

2.2 Meta-heurísticas selecionadas

As meta-heurísticas Recozimento Simulado, Busca Tabu e Algoritmo Genético são clássicas no contexto do VRP, portanto adequadas para o presente estudo. A diversidade técnica entre esses métodos permite utilizar diferentes mecanismos de escape de ótimos locais e estratégias de exploração do espaço de busca diante das restrições de roteamento. A seguir, serão apresentados os fundamentos do funcionamento de cada um deles.

2.2.1 Recozimento Simulado

Explora possíveis soluções por meio de um mecanismo probabilístico inspirado no processo de resfriamento de metais [5]. Se Δf é a diferença de custo entre a nova solução (s') e a atual (s):

$$\Delta f = f(s') - f(s)$$

a probabilidade P de aceitação é definida por:

$$P(\text{aceitar } s') = \begin{cases} 1, & \text{se } \Delta f < 0 \\ e^{-\frac{\Delta f}{T}}, & \text{se } \Delta f \geq 0 \end{cases}$$

onde T é a temperatura, que decai ao longo do tempo conforme uma taxa de resfriamento α :

$$T_{k+1} = \alpha \cdot T_k$$

2.2.2 Busca Tabu

O método utiliza “listas tabu”, que impedem o algoritmo de retornar a soluções visitadas recentemente, forçando uma exploração sistemática e agressiva de novas vizinhanças [6]. Mais objetivamente, o algoritmo opera sobre a vizinhança $N(s)$ da solução atual, mas subtrai os movimentos que estão na Lista Tabu (L):

$$s_{next} = \operatorname{argmin}\{f(s') \mid s' \in N(s) \setminus L\}$$

Entretanto, existe o Critério de Aspiração. Se uma solução s' está na lista tabu, mas seu custo é melhor que a melhor solução encontrada até agora (s_{best}), o estado tabu é ignorado:

$$\text{Se } f(s') < f(s_{best}), \text{ então } s' \in \textit{Candidatos} \text{ mesmo se } s' \in L$$

2.2.3 Algoritmo Genético

Por meio dos operadores de cruzamento, mutação e seleção, o algoritmo é capaz de recombinar sub-rotas eficientes de diferentes soluções para gerar descendentes potencialmente superiores [7, 8]. Isso se traduz em converter o problema de minimização de custos em um valor de sobrevivência, representado pela função de aptidão (*fitness*) $F(s)$:

$$F(s) = \frac{1}{C(s) + \epsilon}$$

onde ϵ é um valor infinitesimal para evitar divisão por zero. Presente, neste caso, apenas por rigor teórico, pois a função de custo que será utilizada é sempre maior que 0. A probabilidade de um indivíduo ser selecionado para o cruzamento é dada por:

$$P_i = \frac{F(s_i)}{\sum_{j=1}^N F(s_j)}$$

onde N é o tamanho da população. Isso garante que rotas mais curtas tenham maior probabilidade biológica de transmitir seus genes (sequências de clientes).

2.3 Sistema multiagente

Os *Multi-Agent Systems* (MAS) consistem em um conjunto de elementos computacionais, denominados agentes, que interagem em um ambiente compartilhado para resolver problemas que ultrapassam as capacidades individuais de um único processador ou algoritmo [9]. Tais sistemas têm sido aplicados com sucesso em diversas áreas, incluindo otimização de colônias de formigas, onde a emergência de comportamentos complexos globais decorre das interações locais entre agentes relativamente simples.

No contexto do VRPTW, a abordagem multiagente é empregada para decompor a complexidade da busca em diferentes especialidades: enquanto um agente foca na exploração global do espaço de busca (*exploration*), outro dedica-se ao refinamento local intensivo (*exploitation*).

Essa arquitetura permite a transição de algoritmos competitivos para um modelo cooperativo, onde a inteligência coletiva emerge da troca de informações entre os agentes. A principal vantagem desta abordagem para o VRPTW é a capacidade de evitar que o sistema fique preso em ótimos locais, uma vez que diferentes comportamentos heurísticos operam simultaneamente e compartilham suas melhores descobertas por meio de uma memória comum.

2.3.1 Framework Mesa

O Mesa é um framework de código aberto desenvolvido em *Python* para a modelagem baseada em agentes, permitindo que pesquisadores criem, manipulem e analisem sistemas complexos compostos por múltiplos elementos autônomos [10].

Diferente de modelos matemáticos puramente estáticos, o Mesa fornece uma estrutura computacional que facilita a observação de comportamentos emergentes a partir das interações locais entre agentes em um ambiente dinâmico.

A arquitetura do Mesa é fundamentada em uma separação clara de responsabilidades, baseada em três componentes principais que seguem padrões de projeto modernos:

- **Agente (*Agent*):** Representa a unidade fundamental de inteligência e ação. Cada agente possui seu próprio estado e lógica interna de decisão, sendo capaz de interagir com o ambiente ou com outros agentes.
- **Modelo (*Model*):** Atua como a entidade de alto nível que orquestra o sistema. O modelo define as regras do ambiente, mantém os dados globais e gerencia a criação e destruição dos agentes.
- **Agendador (*Scheduler*):** É o componente responsável pela coordenação temporal das ações. Ele determina a ordem em que os agentes executam seus passos (*steps*), podendo operar de forma aleatória, sequencial ou simultânea, o que impacta diretamente a dinâmica de convergência do sistema.

No âmbito da otimização combinatória, o Mesa destaca-se por permitir que meta-heurísticas distintas sejam encapsuladas em agentes independentes, transformando um problema de busca individual em um processo de cooperação distribuída.

2.3.2 Estratégia de cooperação: *Path Relinking*

O *Path Relinking* (PR) é uma estratégia de intensificação de busca originalmente proposta como um componente de memória para a Busca Tabu [11], mas que se consolidou como uma técnica poderosa para a integração de soluções em diversos contextos de otimização combinatória.

O conceito fundamental do PR reside na exploração sistemática de trajetórias que conectam duas soluções de alta qualidade, denominadas solução inicial (S_i) e solução guia (S_g). A premissa teórica do método é que, ao navegar pelo caminho que transforma a solução inicial na solução guia, o algoritmo pode descobrir soluções

intermediárias que compartilham características de ambas e que, em muitos casos, superam as referências originais em termos de qualidade.

Esse processo é realizado por meio de uma sequência de movimentos de vizinhança que reduzem progressivamente a distância, comumente medida pela distância de Hamming ou por diferenças estruturais em rotas, entre as duas soluções. Embora o conceito fundamental descreva uma trajetória entre dois pontos, a literatura clássica [11] categoriza o *Path Relinking* em diferentes tipos, dependendo de como e em qual direção essa trajetória é explorada:

- ***Forward Path Relinking* (Direto):** A busca parte da pior solução (Solução Inicial - S_i) em direção à melhor solução (Solução Guia - S_g). A premissa é incorporar progressivamente as boas características da solução guia na solução inicial.
- ***Backward Path Relinking* (Reverso):** A trajetória é percorrida no sentido inverso, partindo da melhor solução (S_g) em direção à pior (S_i). Esta variante costuma apresentar resultados superiores na prática, pois o algoritmo explora a vizinhança imediata da solução de mais alta qualidade logo nos primeiros passos da trajetória.
- ***Mixed Path Relinking* (Misto):** A exploração ocorre de forma alternada, avançando simultaneamente a partir de ambas as extremidades (S_i e S_g) até que os caminhos se encontrem no centro da trajetória, garantindo uma amostragem equilibrada de ambas as vizinhanças.

Para o escopo do modelo de cooperação proposto neste trabalho, a implementação foca especificamente nas estratégias *Forward* e *Backward*, permitindo que os agentes explorem essas trajetórias bidirecionalmente para maximizar a extração de características promissoras das rotas.

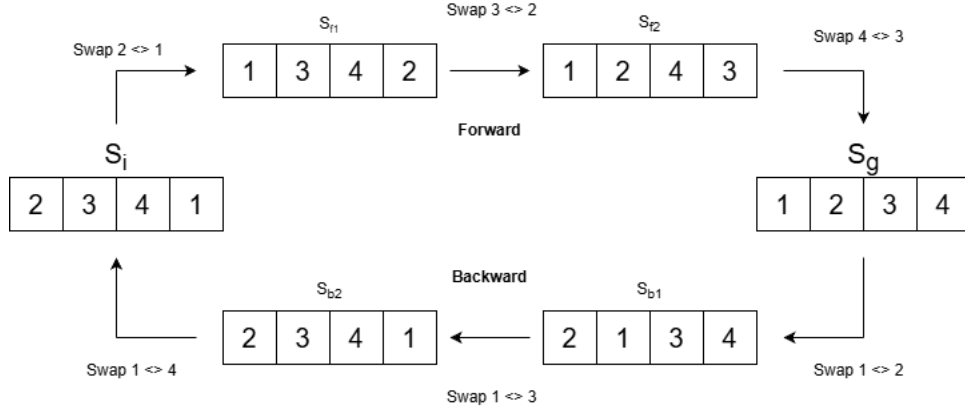


Figura 2.1: Representação do caminho entre S_i e S_g .

No contexto do VRPTW, o *Path Relinking* atua como um mecanismo de cooperação de alto nível ao permitir a sinergia estrutural, combinando a eficiência de sequenciamento de uma solução com a estrutura de particionamento de veículos de outra para criar rotas que nenhum agente isolado seria capaz de conceber. Além disso, a técnica possibilita a exploração de vizinhanças distantes, uma vez que, ao contrário de buscas locais tradicionais que exploram apenas o entorno imediato, o PR investiga uma região específica do espaço de busca delimitada por duas soluções de elite, focando o esforço computacional em áreas comprovadamente promissoras.

Outro fator determinante é a sua capacidade de facilitar o escape de ótimos locais, pois a trajetória entre as soluções pode atravessar vales de soluções inviáveis ou de baixa qualidade para alcançar picos de aptidão mais elevados, agindo como um catalisador para a convergência global do sistema multiagente. Dessa forma, o PR não é apenas um método de busca, mas uma filosofia de cooperação que utiliza o conhecimento acumulado por diferentes agentes especialistas para sintetizar soluções superiores, mitigando as fraquezas individuais observadas em execuções isoladas.

Como seus operadores e manipuladores de soluções são relativamente simples, o PR é menos custoso computacionalmente que uma meta-heurística, seu esforço reside majoritariamente na verificação das funções de custo. Este é seu ponto forte, mas também ponto fraco, uma vez que depende de boas soluções como entrada para melhorá-las ainda mais. É evidente, portanto, que deve ser usada em combinação com outros algoritmos, idealmente ao final das etapas.

2.4 Aprendizado por Reforço

O Aprendizado por Reforço (*Reinforcement Learning* - RL) é uma área do aprendizado de máquina que se preocupa com a forma como agentes inteligentes devem tomar ações em um ambiente a fim de maximizar uma recompensa acumulada. Diferente do aprendizado supervisionado, que se baseia em um conjunto de dados rotulados, fundamenta-se na experiência direta do agente, que aprende por meio de um processo de tentativa e erro ao interagir com o ambiente [12].

A estrutura básica de um problema de RL é frequentemente modelada como um Processo de Decisão de Markov, que envolve os seguintes componentes fundamentais:

- **Agente (*Agent*):** O tomador de decisão que observa o ambiente e executa ações.
- **Ambiente (*Environment*):** O contexto externo com o qual o agente interage.
- **Estado (*State*):** Uma representação da situação atual do ambiente em um determinado instante. No contexto de otimização, o estado pode refletir a qualidade da solução atual, como o número de veículos e a distância total.
- **Ação (*Action*):** O conjunto de decisões possíveis que o agente pode tomar.
- **Recompensa (*Reward*):** Um sinal numérico de *feedback* enviado pelo ambiente após uma ação, que indica o sucesso ou fracasso daquela decisão em relação ao objetivo final.

O objetivo do RL é encontrar uma política (π), que é um mapeamento de estados para ações que maximizem o retorno esperado a longo prazo. Um conceito crítico neste processo é o equilíbrio entre exploração (*exploration*), onde o agente testa novas ações para descobrir sua utilidade, e exploração (*exploitation*), onde o agente utiliza o conhecimento já adquirido para escolher as ações que renderam as maiores recompensas anteriormente.

Um dos mais clássicos algoritmos de RL, O Q-Learning [13] é do tipo *off-policy* e *model-free*, o que significa que ele pode aprender a política ótima de tomada de decisão sem a necessidade de um modelo prévio das transições do ambiente ou das probabilidades de recompensa. O “Q” na nomenclatura refere-se à Qualidade (ou valor de utilidade), representando o quão proveitosa é uma determinada ação em um estado específico.

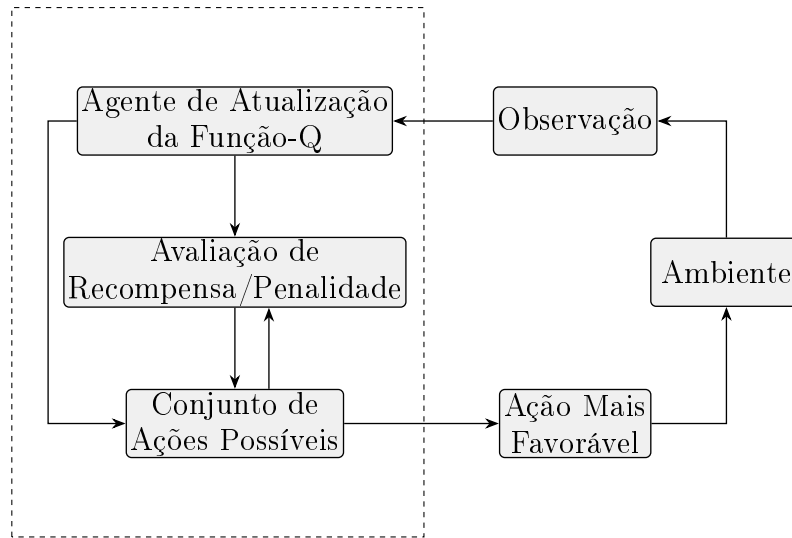


Figura 2.2: Esquema de interação entre agente e ambiente no processo de aprendizagem.

O núcleo do algoritmo consiste em estimar os valores da função $Q(s, a)$, que mensura a recompensa total esperada a longo prazo que um agente pode obter ao realizar a ação a partindo do estado s . Em problemas com espaços de estados discretos, esses valores são armazenados em uma estrutura de dados conhecida como Tabela-Q (*Q-Table*), onde as linhas representam os estados do sistema e as colunas as ações possíveis.

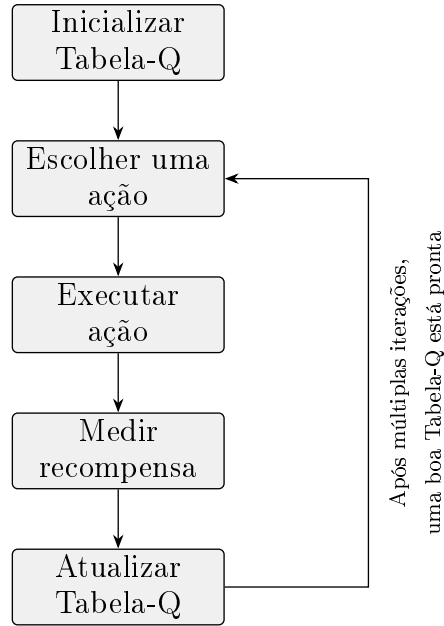


Figura 2.3: Diagrama do algoritmo de atualização do valor- Q .

O aprendizado ocorre através da atualização iterativa dos valores- Q (Q -values) conforme o agente explora o ambiente. Essa atualização fundamenta-se na Equação de Bellman, que ajusta o valor atual com base na recompensa imediata recebida e na melhor estimativa de recompensas futuras:

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)] \quad (2.1)$$

Nesta formulação, os parâmetros de controle desempenham papéis cruciais:

- **Taxa de Aprendizado (α):** Determina o quanto a nova informação substitui o conhecimento antigo.
- **Recompensa Imediata (r):** É o *feedback* numérico direto recebido do ambiente após a execução de uma ação. Representa o ganho instantâneo e serve como a base factual para atualizar a estimativa de qualidade daquela ação.
- **Fator de Desconto (γ):** Define a importância das recompensas futuras em relação às imediatas.
- **Diferença Temporal:** O termo entre colchetes representa o erro de predição, ou seja, a diferença entre o valor que o agente esperava e o que ele efetivamente percebeu após a transição para o novo estado s' .

Para que o algoritmo seja eficaz, o agente deve balancear a busca por novos conhecimentos com o uso do conhecimento já adquirido. A estratégia mais comum é a $\epsilon - greedy$, onde com uma probabilidade ϵ o agente escolhe uma ação aleatória (exploração) e com probabilidade $1 - \epsilon$ ele seleciona a ação com o maior valor-Q atual para aquele estado (exploração). Ao final do processo de treinamento, a Tabela-Q converge para uma representação da estratégia ótima, permitindo que o sistema, ao se deparar com uma determinada configuração do problema, saiba as melhores ações a serem tomadas.

2.5 Avaliação estatística de desempenho algorítmico

A avaliação de desempenho de meta-heurísticas aplicadas a problemas de otimização combinatória, como o VRPTW, impõe desafios metodológicos devido à natureza estocástica inerente a esses algoritmos. A comparação direta de métricas de tendência central, como a média aritmética, é insuficiente para atestar a superioridade de um método sobre outro, uma vez que flutuações amostrais decorrentes de inicializações aleatórias (*seeds*) podem distorcer os resultados. Torna-se imperativo, portanto, o emprego de testes de hipóteses estatísticas.

Visto que as distribuições de custos de rotas e tempos de processamento computacional raramente atendem aos pressupostos clássicos de normalidade (distribuição Gaussiana) e homogeneidade de variância, a aplicação de testes paramétricos tradicionais, como o teste t de Student ou a clássica ANOVA, torna-se inadequada. Em cenários de *benchmarks* múltiplos avaliados sobre um mesmo conjunto de instâncias correlacionadas (amostras pareadas), a literatura prescreve a utilização de abordagens não-paramétricas baseadas na ordenação de resultados por postos (*ranks*).

2.5.1 Teste de Friedman

O Teste de Friedman configura-se como a alternativa não-paramétrica à ANOVA para medidas repetidas. Ele é utilizado para detectar diferenças globais significativas entre múltiplos tratamentos aplicados a diversos blocos de teste. No contexto computacional, os tratamentos correspondem aos diferentes algoritmos ou arquite-

turas implementadas (tais como SA, TS, GA, MAS e MAS-RL), enquanto os blocos representam as instâncias de teste do *benchmark* (como as instâncias de Solomon).

O procedimento do teste consiste em ordenar os algoritmos individualmente dentro de cada instância, atribuindo o posto 1 ao melhor desempenho (ex: menor número de veículos) e o posto k ao pior (sendo k o número total de algoritmos analisados). Em caso de empates numéricos, atribui-se a média dos postos em disputa a cada um dos algoritmos empatados. A hipótese nula (H_0) do teste assume que não há diferenças de desempenho entre os métodos avaliados, implicando que as variações dos *ranks* médios são meramente aleatórias. A hipótese alternativa (H_1) dita que existe uma diferença estatisticamente significativa no desempenho de pelo menos dois algoritmos do grupo.

A estatística do teste (χ_F^2) é computada com base na soma dos postos de cada algoritmo, avaliando a disparidade desses valores em relação ao cenário de equivalência estrita. Se o *p-valor* associado gerado pela distribuição do teste for inferior ao nível de significância preestabelecido θ (tipicamente adotado como 0.05 ou 0.10), rejeita-se a hipótese nula.

2.5.2 Teste Post-Hoc de Nemenyi

A rejeição da hipótese nula no Teste de Friedman estabelece apenas a existência de uma disparidade de desempenho no grupo geral de algoritmos (*omnibus test*), falhando em especificar quais pares exatos de métodos diferem significativamente entre si. Para realizar tais comparações múltiplas par a par, mantendo o controle rigoroso sobre a inflação do Erro do Tipo I (falsos positivos), exige-se a aplicação de um teste *post-hoc*. Neste escopo, o Teste de Nemenyi é a metodologia padrão recomendada para a área de aprendizado de máquina e otimização.

O Teste de Nemenyi determina que o desempenho de dois algoritmos é considerado significativamente distinto se a diferença absoluta entre os seus *ranks* médios superar um limiar matemático rigoroso denominado Diferença Crítica (CD, do inglês *Critical Difference*). O valor numérico da CD é calculado de acordo com a equação:

$$CD = q_\alpha \sqrt{\frac{k(k+1)}{6N}}$$

Onde q_α representa o valor crítico tabelado baseado na distribuição de amplitude *Studentized Range* para o nível de significância α (ou θ) adotado; k representa o número de algoritmos comparados no experimento; e N consiste no número de instâncias ou blocos independentes avaliados.

Graficamente, as conclusões deste teste *post-hoc* são comumente sumarizadas e representadas pelo Diagrama de Diferença Crítica (*Critical Difference Diagram*). Nele, os métodos computacionais são dispostos em um eixo horizontal de acordo com o seu respectivo *rank* médio de desempenho. Algoritmos que não apresentam diferenças estatísticas significativas são agrupados por uma linha espessa de conexão, oferecendo uma percepção visual inequívoca acerca do ranqueamento global, das distâncias de performance e das reais equivalências operacionais entre os sistemas propostos.

Capítulo 3

Método Proposto

Este capítulo descreve a materialização metodológica da pesquisa, mapeando as fases das implementações executadas. A primeira estruturação analítica está descrita na Seção 3.1 com pautas iniciais como pré-processamento de datasets cartesianos, extração dos parâmetros georreferenciados para as estruturas na modelagem computacional e a verificação das rotinas em base conhecida. Em progressão, a Seção 3.2 introduz e apresenta esquematicamente o ambiente multiagente estabelecido através dos componentes unificados baseados no *framework Mesa*, elucidando o comportamento da seleção cooperativa das soluções e o intermédio do *Path Relinking*. O avanço do paradigma do sistema inteligente colaborativo culmina na Seção 3.3 com o controle das execuções repassado para as bases do Q-Learning, estruturado com recompensas dinâmicas, decaimento de aleatoriedade perante convergência nas iterações e gestão estocástica adaptativa.

3.1 Etapas iniciais

As fases preliminares do desenvolvimento focaram na construção da arquitetura base do sistema. Foram implementadas as rotinas de entrada e saída de dados, responsáveis pela leitura dos *datasets* e dos arquivos de solução de referência (*benchmarks*), além de módulos para a visualização gráfica de rotas e monitoramento de desempenho.

Um componente crítico nesta etapa foi a função custo compartilhada, desenvolvida de forma modular para garantir que a avaliação das rotas fosse consistente entre os diferentes algoritmos testados. Para validar a implementação, utilizou-se como referência a **instância RC201** de Solomon (resultado e procedimentos seguidos para alcançar tal, respectivamente em [14, 15]). A figura abaixo apresenta a distribuição espacial dos clientes nesta instância.

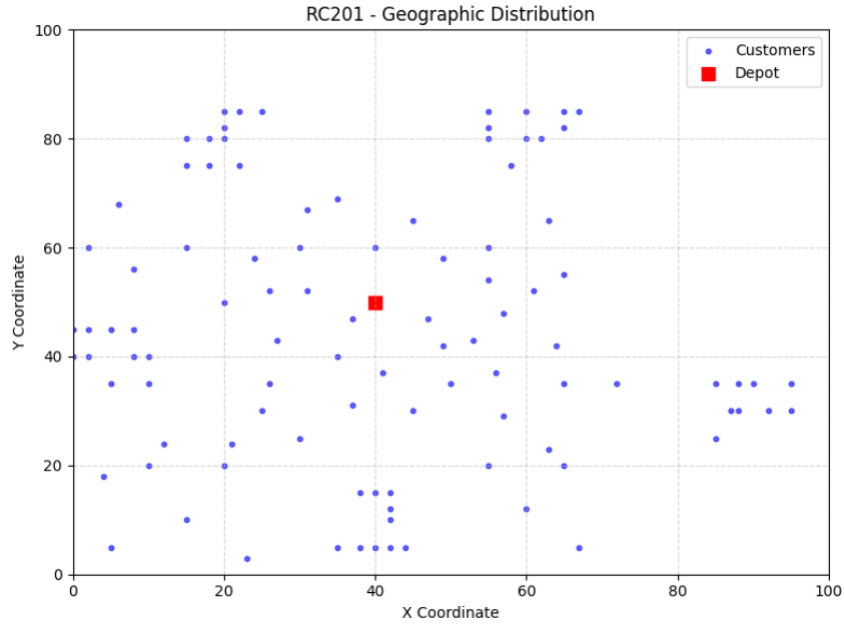


Figura 3.1: Distribuição espacial dos clientes e do depósito na instância RC201 de Solomon.

A validação da função custo ocorreu por meio da leitura da melhor solução conhecida para a RC201. Ao processar essa solução e comparar os resultados obtidos com os indicadores do *benchmark*, confirmou-se a integridade do cálculo de custo. A figura abaixo ilustra o conjunto de rotas ótimas para este cenário.

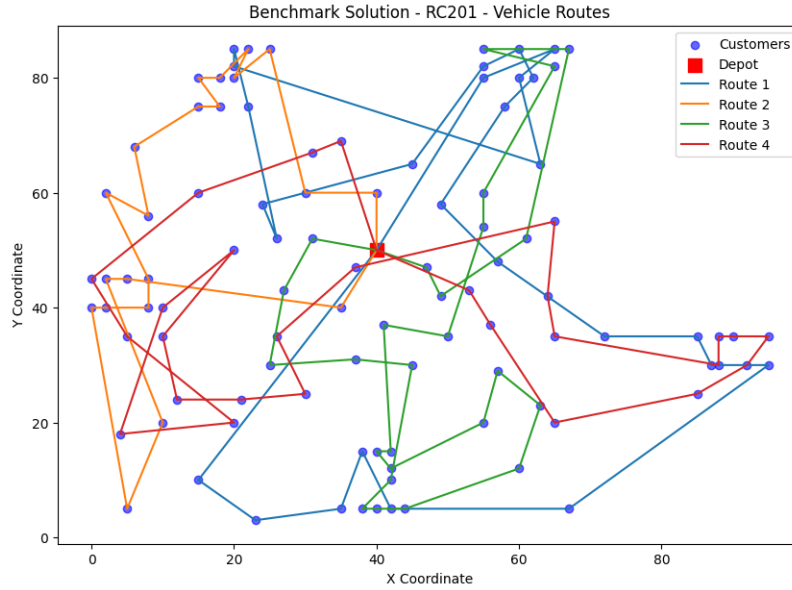


Figura 3.2: Visualização do conjunto de subrotas da melhor solução conhecida para a instância RC201.

Por fim, foram estruturadas as classes base para os algoritmos meta-heurísticos. Essa arquitetura orientada a objetos permitiu a execução dos testes preliminares e a coleta dos primeiros dados para análise comparativa.

Um ponto importante a se notar é a escolha de não separar rotas em viável ou inviável ao longo dos algoritmos. Todas as rotas foram consideradas realizáveis, de forma que a inserção de retorno ao depósito nas rotas se dá pela verificação da impossibilidade de seguir para um próximo cliente, seja pela limitação de carga ou temporal.

3.2 Implementação de sistema multiagente

A arquitetura do sistema proposto foi desenvolvida utilizando o *framework Mesa*, que permite a modelagem de agentes autônomos em um ambiente controlado. No contexto deste trabalho, o MAS atua como uma camada de coordenação que integra as meta-heurísticas independentes, transformando-as em agentes especialistas que colaboram para a otimização do VRPTW.

3.2.1 Arquitetura e componentes do sistema

O modelo é estruturado em três componentes principais, conforme implementado na classe `VRPOptimizationModel`:

- **Agentes Especialistas (*HeuristicAgent*):** Cada instância representa uma meta-heurística (SA, TS ou GA) que atua de forma independente. Em cada ciclo de execução (*step*), o agente recupera uma solução promissora do *pool* de elite para servir como uma solução inicial (evidentemente, em estado melhor que uma solução inicial puramente aleatória), executa sua lógica de busca e devolve o resultado aprimorado ao sistema.
- **Memória Compartilhada (*ElitePool*):** Funciona como um quadro negro que armazena as melhores soluções encontradas por todos os agentes. O *pool* possui um tamanho fixo, garantindo que apenas as soluções com menor custo (considerando o número de veículos K e distância total D) sejam mantidas e compartilhadas. Para garantir variabilidade entre as soluções, também foi implementado mecanismo que evita a inserção de soluções demasiadamente parecidas com as já presentes na *pool*.
- **Mecanismo de Coordenação e Sincronização:** O modelo executa os agentes em paralelo, inserindo, se aptas, suas soluções no *ElitePool* ao final do ciclo.

3.2.2 Cooperação e a estratégia de Path Relinking

A verdadeira cooperação no SMA ocorre através do método `run_path_relinking`. Diferente de uma execução paralela simples, este sistema implementa uma busca cooperativa onde duas soluções de elite são combinadas:

1. **Seleção por Diversidade:** O sistema seleciona a melhor solução atual (*source*) e a compara com outros candidatos no *ElitePool*, escolhendo aquele com maior distância de *Hamming* (maior diversidade) como alvo (*target*).
2. **Exploração Bidirecional:** Adotou-se a estratégia de *Path Relinking* bidirecional, na qual a exploração ocorre simultaneamente em dois sentidos: da solução *source* em direção à *target* e vice-versa. Essa abordagem dobra as trajetórias exploradas entre os dois pontos de elite, permitindo que o algoritmo

investigue vizinhanças distintas que poderiam ser negligenciadas em uma busca unidirecional.

3. **Permutação e Reavaliação:** Durante a trajetória, a técnica gera caminhos intermediários permutando clientes e reavaliando a função de custo a cada passo. O uso do critério bidirecional acelera o encontro de soluções que compartilham características de ambos os genitores de elite.
4. **Sintetização de Soluções:** Se uma solução intermediária superar as originais em termos de custo (veículos K ou distância D), ela é adicionada ao *pool*, permitindo que o sistema escape de ótimos locais ao fundir características de rotas distintas geradas por agentes diferentes.

O fluxo operacional do MAS está sintetizado na Figura 3.3.

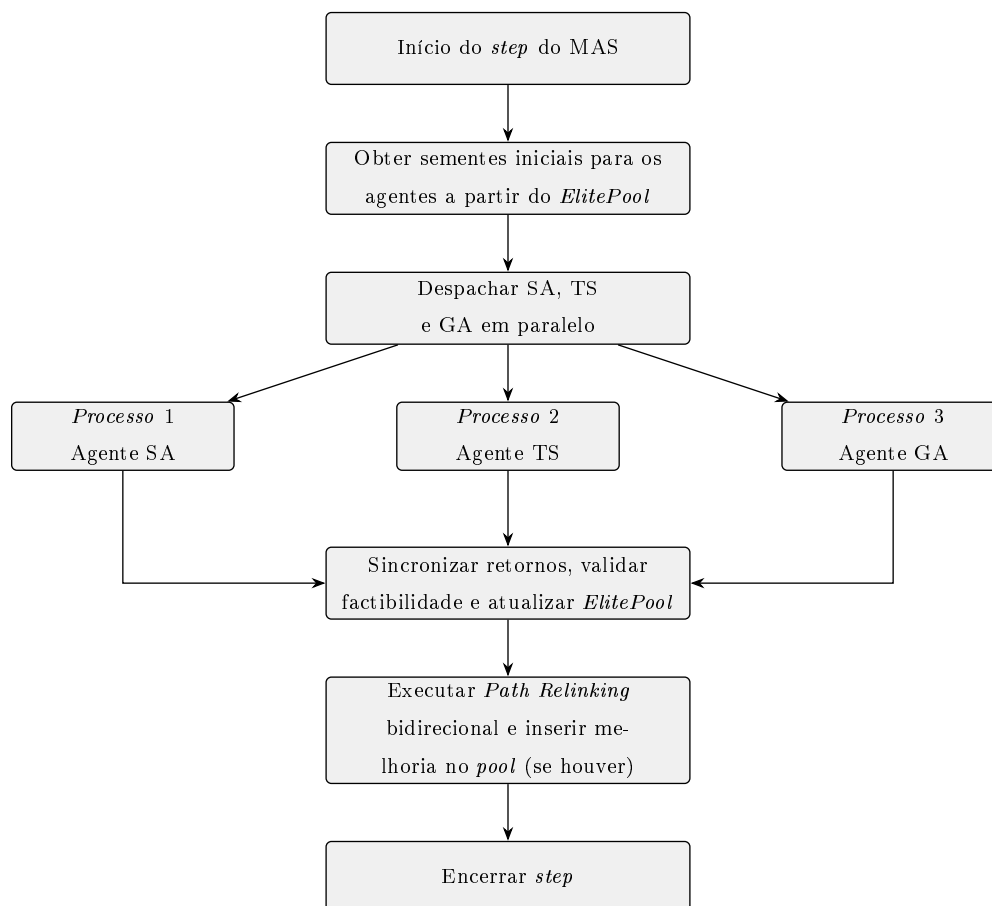


Figura 3.3: Fluxo de execução do VRPOptimizationModel na configuração final paralela. O *step* executa SA, TS e GA em paralelo, consolida soluções no *ElitePool* e aplica o *Path Relinking* sequencialmente antes do encerramento.

Uso de processos x threads

Ressalta-se a limitação fundamental no uso de *threads* para a paralelização em Python, que reside na presença do *Global Interpreter Lock (GIL)*. O *GIL* é um mecanismo de exclusão mútua que assegura que apenas uma *thread* execute *bytecode* Python por vez, independentemente da quantidade de núcleos de processamento disponíveis na arquitetura de hardware utilizada.

Em tarefas *CPU-bound*, nas quais o processamento é predominantemente matemático e intensivo, como é o caso das meta-heurísticas aplicadas, a utilização de *threads* não resulta em paralelismo efetivo. Pelo contrário, o *GIL* força a alternância de contexto (*context switching*) entre as *threads*, introduzindo um *overhead* que pode degradar o desempenho global do sistema.

Para contornar esta restrição, a arquitetura deste trabalho adota o *ProcessPoolExecutor*, que fundamenta-se na utilização de processos independentes. Diferente das *threads*, cada processo possui seu próprio interpretador Python, seu próprio espaço de memória isolado e, crucialmente, seu próprio *GIL*. Desta forma, o Sistema Operacional é capaz de escalar cada processo para um núcleo de CPU distinto, garantindo o paralelismo real e permitindo que os agentes operem de forma simultânea e eficiente durante a busca por soluções.

Isto exige uma estrutura de comunicação entre processos, sincronização dos retornos e controle mais manual da memória, mas os ganhos em tempo superam significativamente os custos.

3.3 Hiper-heurística baseada em Aprendizado por Reforço

A evolução do sistema multiagente convencional para o modelo proposto reside na implementação da classe `VRPOptimizationModelRL`. Neste modelo, o mecanismo de coordenação deixa de executar sempre uma busca para cada agente e passa a

utilizar o algoritmo *Q-Learning* para orquestrar a ativação dos agentes especialistas, configurando uma abordagem de hiper-heurística baseada em aprendizado [16].

Modelagem do Q-Learning

O agente de RL opera sobre um ciclo de percepção e ação, onde a política de decisão é refinada com base na experiência acumulada no ambiente de busca. Os componentes do modelo são definidos como segue:

- **Espaço de Estados (S):** O estado é definido pela tupla discreta (K, D_{bin}) . Para evitar a explosão do espaço de estados e garantir a convergência da Tabela-Q, a distância total (D) é discretizada em faixas (*bins*) de unidades através do parâmetro `distance_bin_size`.
- **Espaço de Ações (A):** Corresponde à seleção de uma das três meta-heurísticas disponíveis para execução no ciclo atual: *Simulated Annealing* (SA), *Tabu Search* (TS) ou *Genetic Algorithm* (GA).
- **Função de Recompensa Hierárquica (R):** A recompensa é estruturada para refletir a prioridade multiobjetivo do *VRPTW*:
 - **Redução de Frota:** Atribui-se uma recompensa massiva caso a ação resulte na diminuição do número de veículos (K).
 - **Redução de Distância:** A recompensa é equivalente à magnitude da melhoria em D .
 - **Penalidade por Estagnação:** Aplica-se uma pequena penalidade se a ação não gerar melhorias, incentivando a exploração de outros agentes.
- **Política de Seleção:** Utiliza-se a estratégia *ϵ -greedy*, equilibrando a exploração do conhecimento atual com a exploração de novas trajetórias de busca. Além disso aplica-se um decaimento progressivo de ϵ ao longo dos ciclos para favorecer a convergência.

A cada *step* do modelo, o sistema realiza três iterações de escolha e execução. Após a execução do agente selecionado, o valor de utilidade $Q(s, a)$ é atualizado na Tabela-Q utilizando a Equação 2.1. Paralelamente ao RL, o método `run_path_relinking`

continua a promover a síntese de soluções de elite, agora alimentado por um *pool* cujos candidatos são gerados por agentes selecionados de forma adaptativa.

No método final implementado (classe `VRPOptimizationModelRL`), cada *step* ocorre em duas fases. Na fase de decisão, o agente de RL seleciona `runs_per_step` ações por política ϵ -greedy. Na fase de execução, as ações selecionadas são despachadas em paralelo. Após a sincronização dos processos, cada solução retornada é avaliada, inserida no *ElitePool* (quando viável), e o valor de utilidade $Q(s, a)$ é atualizado conforme a Equação 2.1. Este funcionamento é ilustrado no fluxograma da Figura 3.4.

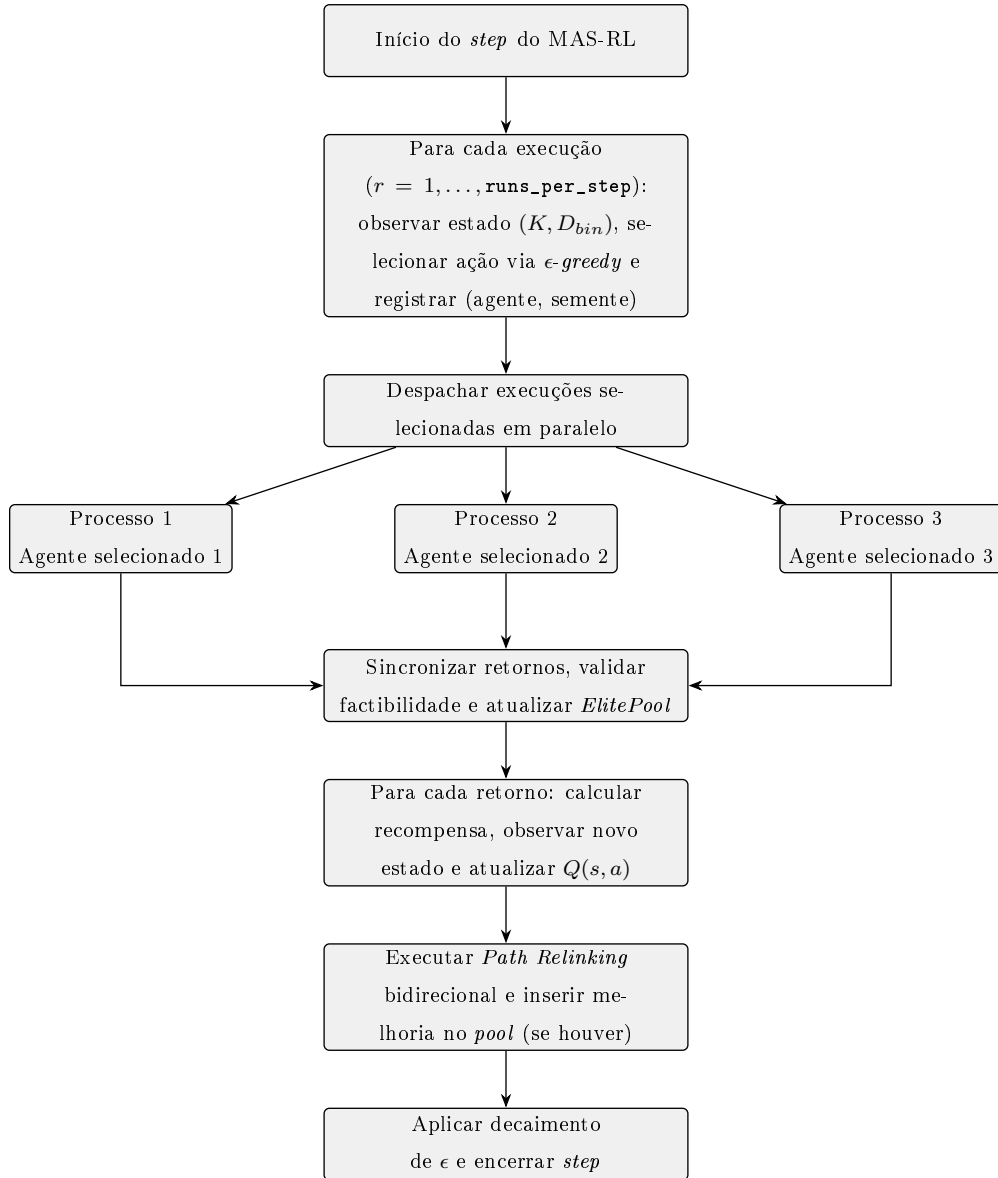


Figura 3.4: Fluxo de execução do método final implementado em `VRPOptimizationModelRL`. A seleção de ações é feita pelo agente de RL, enquanto as execuções das meta-heurísticas são despachadas em paralelo. A atualização da Tabela-Q ocorre após a sincronização dos retornos, seguida pela aplicação do *Path Relinking* e o decaimento de ϵ .

Abaixo, o um recorte do registro de eventos de uma execução do MAS-RL ilustra o comportamento e evolução a cada passo.

```

Running MAS_RL (rerun) on c101.txt
MAS run=1 RL Step 1
==> State (999, 999) -> Run 1 selected SA, Run 2 selected TS, Run 3 selected TS
    |-- Run 1 agent SA result -> vehicles: 21, distance: 2094.51
  
```

```

|   Update: Reward=0.00 | Q(SA) of state(999, 999) updated to 0.00
|-- Run 2 agent TS result -> vehicles: 27, distance: 2313.44
|   Update: Reward=-0.05 | Q(TS) of state(999, 999) updated to -0.01
'-- Run 3 agent TS result -> vehicles: 27, distance: 2313.44
    Update: Reward=-0.05 | Q(TS) of state(999, 999) updated to -0.02
==> PR start: source (k=21, d=2094.51) -> target (k=27, d=2313.44), diversity=100
PR candidates -> forward (k=21, d=2094.51), reverse (k=21, d=2094.51)
PR finished: no improvement over source

MAS run=1 RL Step 2
==> State (21, 20) -> Run 1 selected SA, Run 2 selected TS, Run 3 selected SA
|   Run 1 agent SA result -> vehicles: 20, distance: 1837.20
|   Update: Reward=457.32 | Q(SA) of state(21, 20) updated to 91.46
|   Run 2 agent TS result -> vehicles: 25, distance: 1959.95
|   Update: Reward=-0.05 | Q(TS) of state(21, 20) updated to -0.01
'-- Run 3 agent SA result -> vehicles: 23, distance: 2223.47
    Update: Reward=-0.05 | Q(SA) of state(21, 20) updated to 73.16
==> PR start: source (k=20, d=1837.20) -> target (k=21, d=2094.51), diversity=100
PR candidates -> forward (k=20, d=1837.20), reverse (k=20, d=1837.20)
PR finished: no improvement over source

...

MAS run=1 RL Step 20
==> State (18, 14) -> Run 1 selected TS, Run 2 selected TS, Run 3 selected GA
|   Run 1 agent TS result -> vehicles: 18, distance: 1496.46
|   Update: Reward=-0.05 | Q(TS) of state(18, 14) updated to -0.01
|   Run 2 agent TS result -> vehicles: 18, distance: 1486.70
|   Update: Reward=9.69 | Q(TS) of state(18, 14) updated to 1.93
'-- Run 3 agent GA result -> vehicles: 18, distance: 1570.27
    Update: Reward=-0.05 | Q(GA) of state(18, 14) updated to 0.27
==> PR start: source (k=18, d=1486.70) -> target (k=18, d=1496.39), diversity=19
PR candidates -> forward (k=18, d=1486.70), reverse (k=18, d=1478.29)
PR improved elite pool -> (k=18, d=1478.29)

```

Este registro evidencia uma pequena desvantagem da aplicação do paralelismo em um ambiente de aprendizado por reforço. A cada *step*, o agente de RL seleciona múltiplas ações que são executadas em paralelo. No entanto, a atualização da Tabela-Q só ocorre após a sincronização dos retornos, o que significa que o agente de RL não tem a oportunidade de ajustar sua política entre as execuções paralelas dentro do mesmo *step*.

Capítulo 4

Experimentos Computacionais

Este capítulo detalha o protocolo experimental adotado para a avaliação das meta-heurísticas clássicas e dos sistemas multiagente propostos. Inicialmente, a Seção 4.1 especifica os recursos computacionais e os pacotes de software utilizados, garantindo a reprodutibilidade dos experimentos. Na sequência, a Seção 4.2 apresenta as instâncias do *benchmark* de Solomon selecionadas, enquanto a Seção 4.3 descreve os hiperparâmetros e os detalhes de implementação que regem cada estratégia de busca. Por fim, a Seção 4.4 discute os resultados obtidos sob duas perspectivas: uma análise global agregada, que utiliza métricas de *gap* normalizado para comparar a eficácia das abordagens, e uma análise individualizada por instância, que ilustra a consistência da superioridade das estratégias multiagente em diversos cenários logísticos.

4.1 Software e hardware

Uma vez que o tempo de processamento é relevante para a análise deste trabalho, serão especificadas as configurações do ambiente nos quais os testes foram realizados, bem como o código fonte utilizado, permitindo a reprodutibilidade dos resultados.

```
Sistema Operacional: Windows 11 Home 64 bits (10.0, build 22631)
Fabricante do Sistema: Dell Inc.
Modelo do Sistema: Inspiron 14 5435
BIOS: 1.17.0
Processador: AMD Ryzen 7 7730U com Radeon Graphics (16 CPUs) - 2.0GHz
Memoria: 16384MB RAM (16GB)
```

O trabalho foi desenvolvido com **Python v3.11.9**, cujos principais módulos utilizados, bem como suas versões, estão listados abaixo:

<code>matplotlib</code>	<code>3.10.6</code>
<code>Mesa</code>	<code>3.3.1</code>
<code>numpy</code>	<code>2.3.3</code>
<code>pandas</code>	<code>2.3.2</code>

O código fonte se encontra em: <https://github.com/hugodba/TCC>

4.2 Instâncias de teste

Neste trabalho, os experimentos numéricos são realizados utilizando um conjunto diversificado de instâncias, englobando diferentes disposições geográficas, características de janelas de tempo e capacidades de carga. Especificamente, o conjunto de testes é composto por 12 instâncias, escolhidas de forma a constituírem um conjunto representativo: *c101*, *c102*, *c201*, *c202*, *r101*, *r102*, *r201*, *r202*, *rc102*, *rc103*, *rc201* e *rc204*. Detalhes adicionais sobre os parâmetros operacionais de cada cenário podem ser verificados no Apêndice A.

4.3 Detalhes de implementação e parametrização

Dado que a meta-heurísticas podem continuar executando por tempo indefinido, os seus hiperparâmetros foram escolhidos balizados pela performance e estagnação. Ou seja, ao longo de testes preliminares, buscou-se um resultado ótimo e o encerramento da execução o mais rápido possível após atingi-lo. Os hiperparâmetros globais de controle do MAS e do RL também buscaram resultados ótimos, porém sem a preocupação de limitar o tempo de execução quando estagnado.

Para reprodutibilidade, com o intuito de mitigar a aleatoriedade inerente às meta-heurísticas, foi implementado o uso de *seeds* fixas nos geradores de números aleatórios. Tal procedimento assegura que os experimentos sejam determinísticos, permitindo a comparação direta entre diferentes rodadas e configurações. Além disso, para obtenção de maior validade estatística, dada a existência inerente de elementos

estocásticos nos métodos propostos, cada algoritmo foi executado 10 vezes para cada instância e foram levados em consideração suas médias de resultado nas análises e comparações.

Quanto à instrumentação dos testes, adotou-se, para mensuração do tempo, a função `time.process_time()`, que contabiliza exclusivamente o tempo de CPU dedicado ao processo, ignorando períodos de ociosidade (*sleep*) e operações de entrada/saída (I/O). Essa escolha garante que a métrica de desempenho reflita mais fielmente o esforço computacional do algoritmo.

Tabela 4.1: Parâmetros de implementação e controle dos experimentos.

Categoria	Parâmetro	Valor
Controles	Número de rodadas	10
	Seed inicial	1
MAS	Tamanho do <i>Pool</i>	3
	Limiar de diversidade	10
	Passos por ciclo (N_STEPS)	20
	Política de escolha	Aleatória
Meta-heurísticas	SA (Temp. Inicial, α , Min, Iters)	1800, 0.99, 3, 275
	TS (<i>Max Iters</i> , <i>Tenure</i> , <i>Samples</i>)	480, 20, 350
	GA (Pop., Gen., P_{cross} , P_{mut})	70, 2300, 0.7, 0.05
Meta-heurísticas (MAS)	SA (Temp. Inicial, α , Min, Iters)	1800, 0.99, 3, 28
	TS (<i>Max Iters</i> , <i>Tenure</i> , <i>Samples</i>)	80, 20, 180
	GA (Pop., Gen., P_{cross} , P_{mut})	70, 210, 0.7, 0.05
Q-Learning	ϵ (Inicial / Mínimo)	0.8 / 0.05
	Decaimento de ϵ	0.992
	α (Taxa de aprendizado)	0.2
	γ (Fator de desconto)	0.8
	Bin de distância	100
	Execuções por <i>step</i>	3

4.4 Resultados dos experimentos

Nesta seção, serão apresentados e analisados os resultados obtidos a partir dos experimentos conduzidos, com foco na comparação entre as meta-heurísticas puras e o sistema multiagente proposto. A análise será estruturada em torno de métricas de desempenho relevantes, como o número de veículos utilizados (K) e o tempo de execução, além de considerar a evolução das soluções ao longo do tempo e a estabilidade dos resultados. Como objetivo secundário, a distância total percorrida (D) também será avaliada, porém com menor ênfase. Os dados brutos de retorno dos experimentos pode ser encontrado no Apêndice C.

Desempenho global

Para possibilitar uma análise comparativa equitativa e agregada entre instâncias com ordens de grandeza de frotas e distâncias marcadamente distintas, aplicou-se um procedimento de normalização adimensional sobre os resultados brutos. A métrica adotada fundamenta-se no cálculo do *gap* relativo percentual em relação às melhores soluções conhecidas na literatura (*Best Known Solutions* — BKS), conforme estabelecido pelas Equações a seguir:

$$\text{Gap } K = \frac{K_{\text{médio}} - K_{\text{ref}}}{K_{\text{ref}}}$$

$$\text{Gap } D = \frac{D_{\text{médio}} - D_{\text{ref}}}{D_{\text{ref}}}$$

Onde $K_{\text{médio}}$ e $D_{\text{médio}}$ representam as médias obtidas pelo algoritmo ao longo das 10 rodadas experimentais, enquanto K_{ref} e D_{ref} correspondem aos valores de referência de Solomon. Sob essa ótica, um *gap* igual a zero indica o alcance exato da melhor solução da literatura. Essa metodologia viabiliza o agrupamento macro de todas as 120 execuções do lote de testes em distribuições estatísticas unificadas.

O comportamento agregado dessa normalização para a frota final de veículos (K) e para a distância total percorrida (D), considerando todos os cenários combinados, pode ser observado nos diagramas de caixa apresentados na Figura 4.1 e na Figura 4.2, respectivamente.

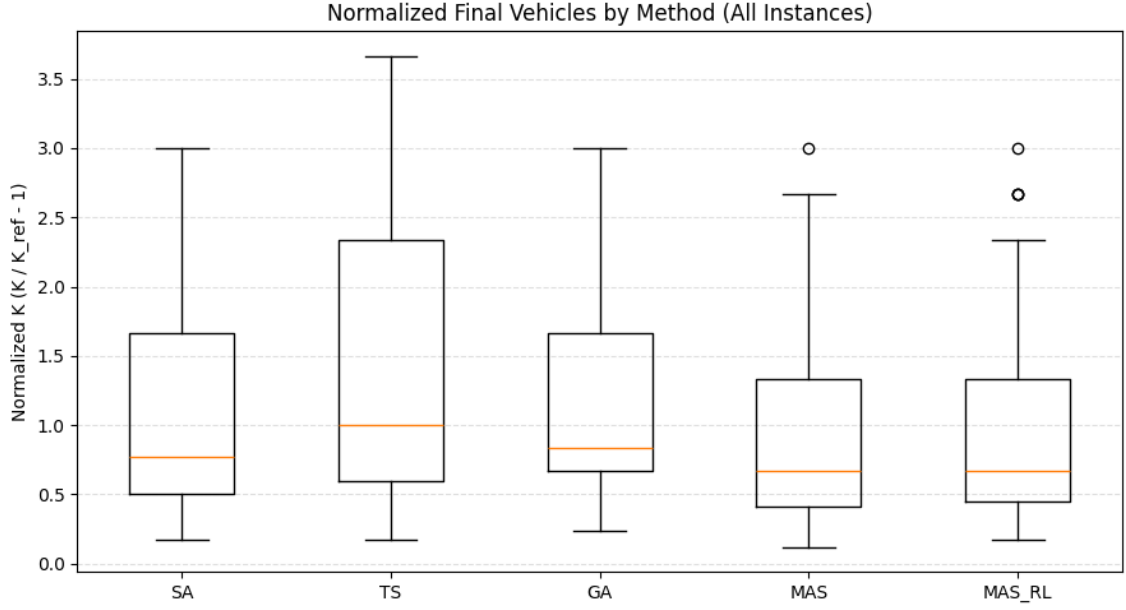


Figura 4.1: Distribuição do gap normalizado da frota final (K) por método para todas as instâncias.

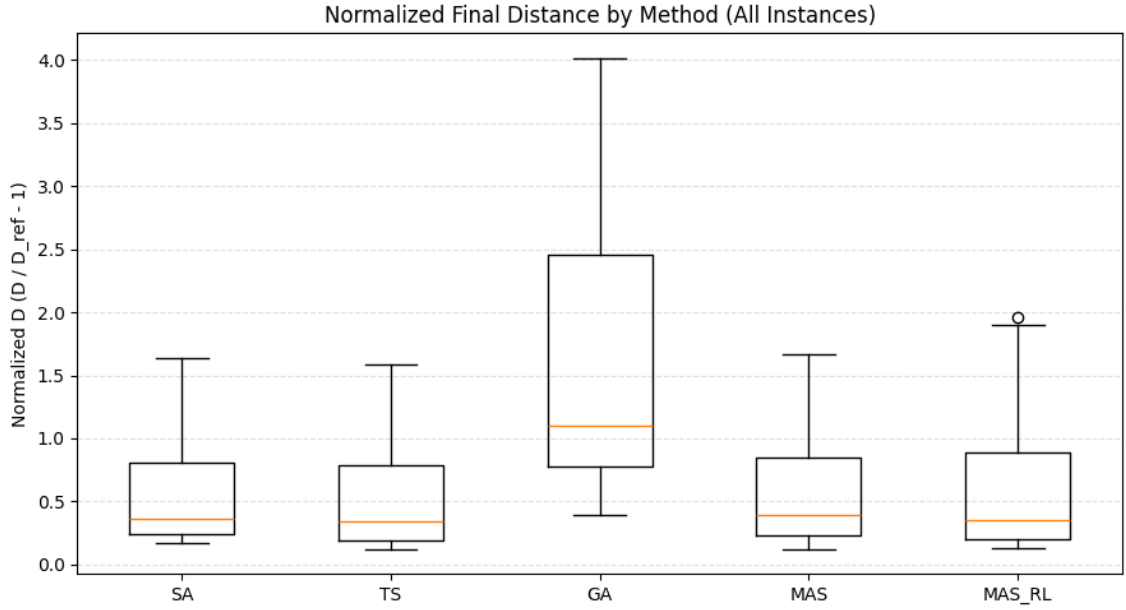


Figura 4.2: Distribuição do gap normalizado da distância final (D) por método para todas as instâncias.

A análise macro das distribuições revela uma clara divisão de desempenho entre as abordagens tradicionais e as arquiteturas baseadas em cooperação. Ao avaliar o comportamento puramente agregado do lote completo de testes, os métodos estruturados em agência — *MAS* (Sistema Multiagente) e *MAS-RL* (Sistema Multiagente

com Aprendizado por Reforço) — demonstram uma superioridade global evidente em relação às meta-heurísticas isoladas. Essa vantagem se traduz visualmente pelo deslocamento descendente de toda a estrutura dos diagramas de caixa, indicando uma tendência consistente de maior eficiência na minimização dos objetivos quando os solucionadores operam de forma cooperativa.

Conforme evidenciado no diagrama de frota (Figura 4.1), a mediana do *gap* de veículos para os frameworks *MAS* e *MAS-RL* consolida-se em patamares inferiores a 0.7, enquanto abordagens consolidadas como a Busca Tabu (*TS*) e o Algoritmo Genético (*GA*) operam com valores centrais próximos ou superiores a 1.0. Mais do que a redução do erro médio, as arquiteturas multiagente encolheram significativamente o intervalo interquartil da distribuição. Esse achatamento da caixa prova que o ecossistema cooperativo mitiga o risco de o algoritmo ficar aprisionado em mínimos locais medíocres ao usar as forças de cada solucionador, atingindo resultados mais consistentes, menos variáveis.

No que tange à dinâmica de convergência, e não apenas ao resultado final, os métodos de agência também se destacam. A evolução temporal do *gap* de frota revela que os sistemas multiagente não apenas alcançam soluções de melhor qualidade, mas o fazem de maneira mais rápida e consistente ao longo do tempo.

Devido à natureza estocástica e aos tempos de execução variados entre as diferentes instâncias e rodadas, a construção dos gráficos com componente de tempo agregou os dados por meio de uma reamostragem temporal. Primeiramente, as trajetórias de cada método foram alinhadas em um *grid* uniforme de 200 pontos temporais entre o instante inicial e o tempo máximo de execução, utilizando interpolação de degrau para preservar a natureza discreta das atualizações de frota.

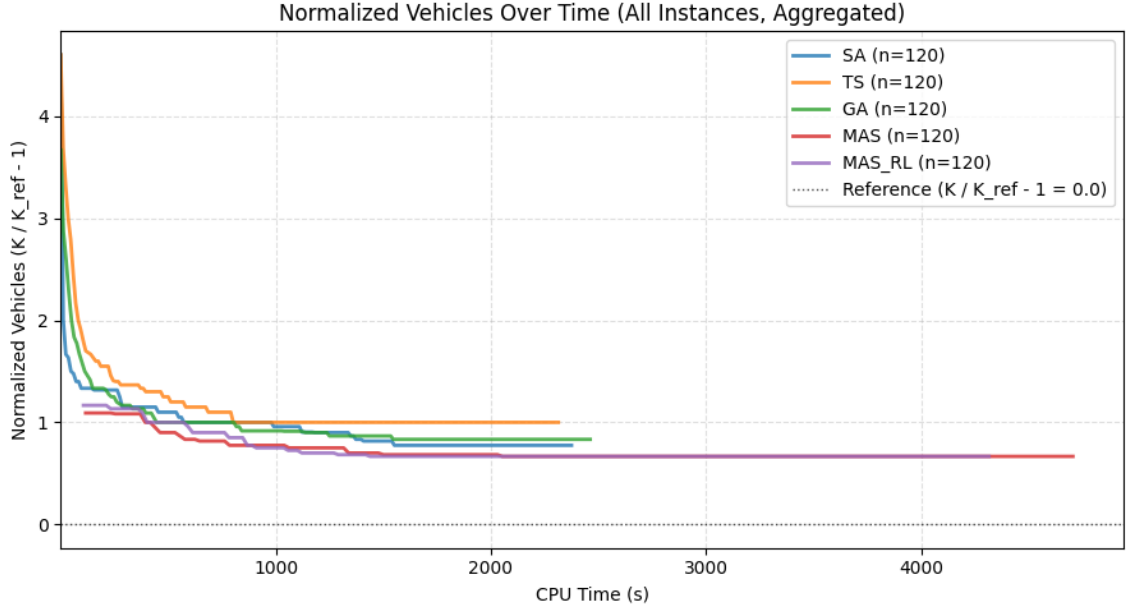


Figura 4.3: Evolução temporal do gap de frota (K) para todas as instâncias combinadas.

É relevante observar que, desde o instante em que iniciam sua trajetória no gráfico, as abordagens multiagente posicionam-se em patamares de *gap* significativamente inferiores aos métodos tradicionais. Esse comportamento decorre da natureza cooperativa do sistema: enquanto as meta-heurísticas puras são projetadas para uma convergência gradual ao longo de uma execução longa, os agentes do *MAS* buscam resultados ótimos em cada um de seus steps, que são mais curtos. Além disso, eles partem de soluções iniciais enriquecidas promovidas pelo *Path Relinking*. Desta forma, a arquitetura proposta mitiga a tradicional compensação (*trade-off*) onde uma convergência rápida seria acompanhada por um desempenho final inferior, demonstrando que a integração entre agentes e memória compartilhada permite atingir resultados de alta qualidade de forma célere e sustentada.

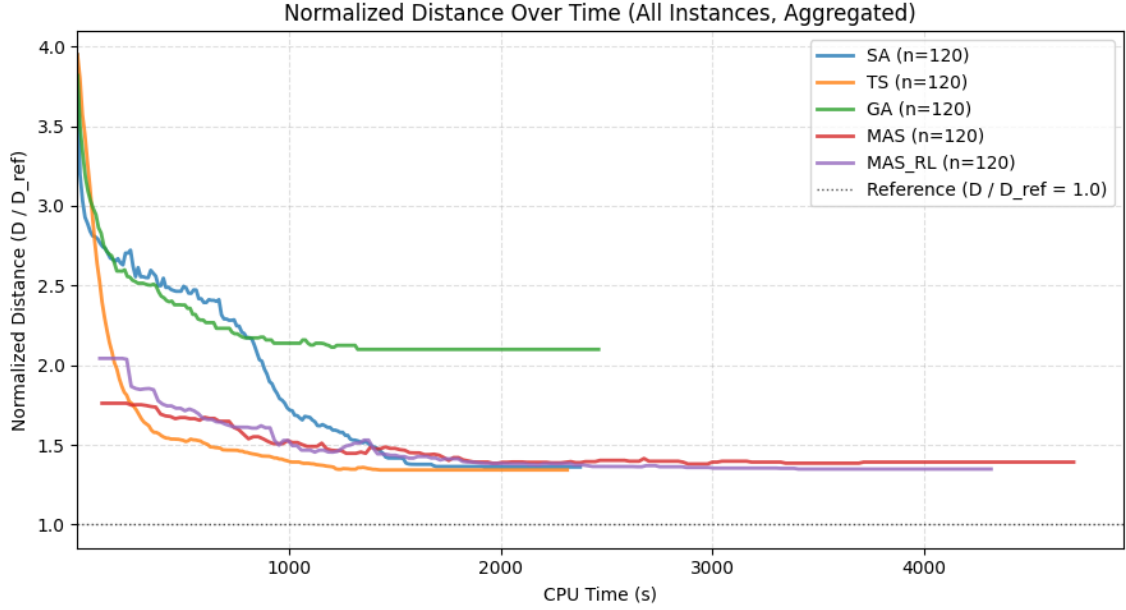


Figura 4.4: Evolução temporal do gap de distância (D) para todas as instâncias combinadas.

Note que as linhas referentes a *MAS* e *MAS-RL* começam em um tempo maior, devido à arquitetura de implementação que recolhe as soluções parciais apenas após o término de cada *step*, diferente das meta-heurísticas puras que atualizam a solução a cada iteração.

É imperativo destacar, contudo, a existência de um cenário de equivalência estatística primária ao comparar os resultados de qualidade de solução entre o *MAS* puro e o *MAS-RL*. Ambas as distribuições exibem caixas quase sobrepostas nos gráficos 4.1 e 4.2, limites de dispersão equivalentes e medianas virtualmente idênticas no que tange ao *gap* de frota e distância. A discussão da causa deste comportamento será detalhada adiante nas subseções subsequentes.

Desempenho por instâncias

A análise detalhada por instância, consolidada na Figura 4.5, reforça a superioridade das abordagens baseadas em sistemas multiagente em relação às meta-heurísticas isoladas. Observa-se que, independentemente da topologia da instância (tipo *C*, *R* ou *RC*) ou do nível de restrição (Séries 1 e 2), as estratégias propostas

demonstram um desempenho superior ou, no mínimo, equivalente às linhas de base tradicionais (SA , GA , TS).

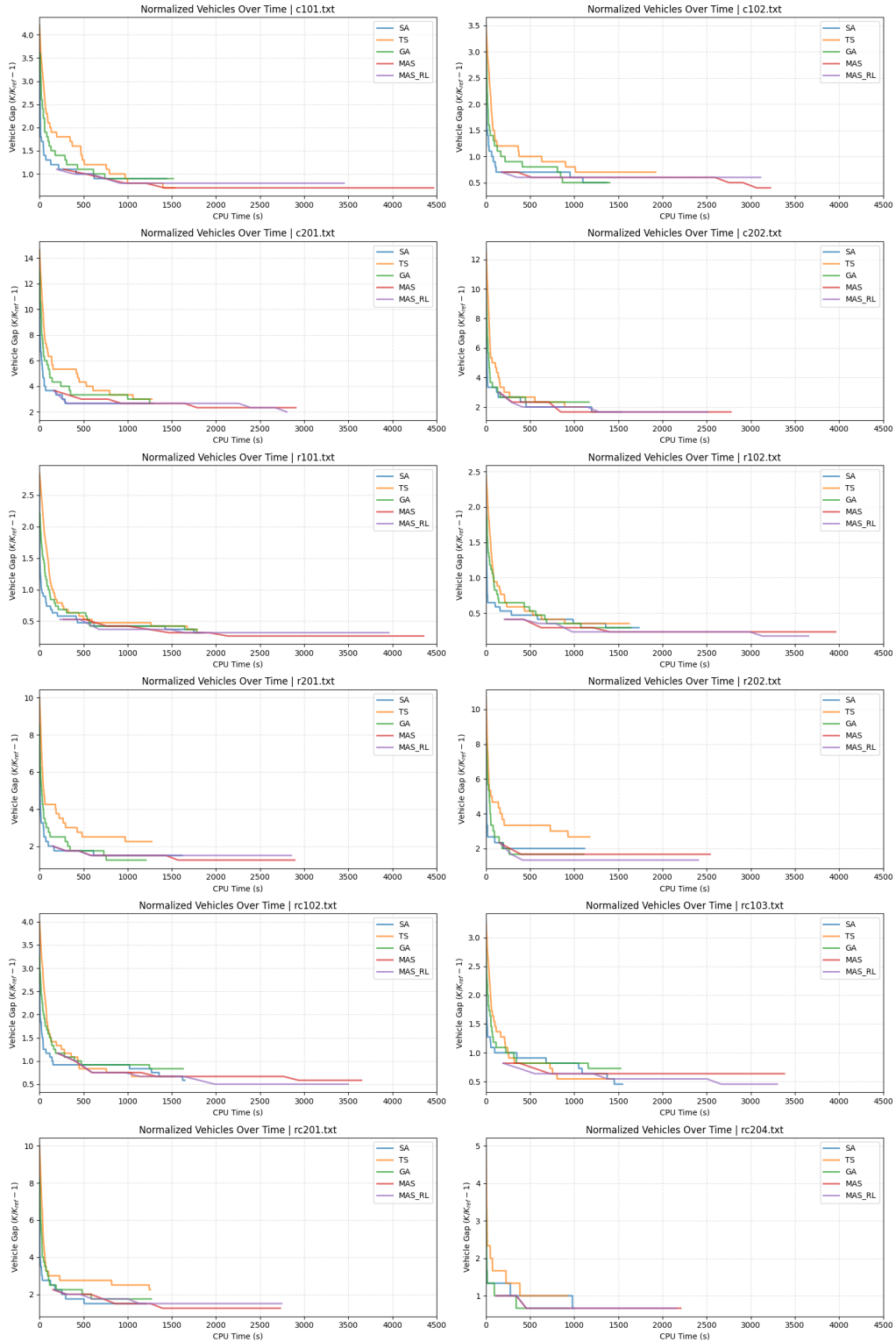


Figura 4.5: Análise de convergência por instância. As curvas ilustram o comportamento dos métodos em diferentes topologias, evidenciando a superioridade do MAS e $MAS-RL$ na exploração do espaço de busca.

4.5 Análise estatística

Um teste de hipóteses foi conduzido com o objetivo de determinar se as diferenças de desempenho observadas entre os métodos propostos possuem significância estatística. Uma vez que os dados de desempenho computacional raramente seguem uma distribuição normal em otimização estocástica, e considerando que as medições foram realizadas de forma pareada nas mesmas instâncias de Solomon, abordagens não-paramétricas foram aplicadas.

Neste estudo, utilizou-se um nível de significância de $\theta = 0.05$. O procedimento estatístico foi dividido em duas etapas. Inicialmente, aplicou-se o Teste de Friedman para avaliar a existência de diferenças globais no desempenho dos cinco algoritmos testados. As hipóteses formuladas para o Teste de Friedman foram:

- Hipótese Nula (H_0): Não existe diferença significativa entre as médias dos resultados obtidos pelos métodos avaliados; todos possuem desempenhos estatisticamente equivalentes.
- Hipótese Alternativa (H_1): Existe diferença significativa entre pelo menos dois dos algoritmos testados.

O Teste de Friedman reportou uma estatística $\chi^2 = 41.48$ e um p-valor associado de 2.13×10^{-8} . Como o p-valor $\ll \theta$, a hipótese nula H_0 é fortemente rejeitada, confirmando que há discrepância significativa na eficácia dos algoritmos perante as instâncias analisadas. Os resultados agregados que basearam este teste estão sumarizados na Tabela 4.2.

Comprovada a diferença global, procedeu-se à segunda etapa da análise através do Teste de Nemenyi, um teste *post-hoc* utilizado para identificar empiricamente, par a par, quais algoritmos apresentam diferenças significativas entre si. A Tabela 4.3 sumariza os *ranks* médios atribuídos a cada algoritmo, onde menores valores indicam maior aderência aos melhores resultados de otimização, bem como a matriz de p-valores das comparações pareadas.

Tabela 4.2: Matriz de Resultados Agregados (Média de Veículos por Instância)

Instância	SA	TS	GA	MAS	MAS-RL
c101	18.2	18.8	19.1	17.0	17.1
c102	15.2	17.0	16.5	15.0	14.8
c201	11.1	12.1	11.3	10.1	10.2
c202	8.5	10.6	9.2	7.6	7.7
r101	25.2	25.5	26.7	24.0	24.0
r102	21.1	21.6	22.5	20.1	20.5
r201	9.3	12.5	9.8	9.1	9.0
r202	7.8	11.1	7.8	7.2	7.4
rc102	19.0	20.4	21.4	18.1	18.0
rc103	16.3	17.7	18.7	16.3	16.2
rc201	10.5	13.2	10.5	9.5	9.4
rc204	5.2	5.8	5.0	4.6	4.6

Tabela 4.3: Matriz de p-valores do Teste Post-Hoc de Nemenyi e ranks médios

Método	Rank Médio	p-valor vs. Algoritmo				
		MAS-RL	MAS	SA	GA	TS
MAS-RL	1.50	-	1.0000	0.0867	0.0002	0.0000
MAS	1.54	1.0000	-	0.1016	0.0003	0.0000
SA	3.12	0.0867	0.1016	-	0.4076	0.1583
GA	4.25	0.0002	0.0003	0.4076	-	0.9858
TS	4.58	0.0000	0.0000	0.1583	0.9858	-

Os testes indicam uma vantagem estatística notória dos sistemas multiagentes. O modelo *MAS-RL* atingiu o menor *rank* médio (1.50), intimamente seguido pelo modelo cooperativo puro MAS (1.54). Em contraste, as meta-heurísticas isoladas obtiveram ranks substancialmente maiores (SA: 3.12; GA: 4.25; TS: 4.58).

A matriz de p-valores gerada pelo Teste de Nemenyi (Tabela 4.3) comprova que o *MAS-RL* supera significativamente tanto o TS (p-valor = 0.0000) quanto o GA (p-valor = 0.0002). Destaca-se também que, para a métrica de veículos, a diferença entre o *MAS-RL* e o SA (p-valor = 0.0867) encontra-se no limite tendencial, não rejeitando H_0 estritamente sob $\theta = 0.05$. Por fim, os resultados do *MAS* e do *MAS-RL* (p-valor = 1.0000) são considerados estatisticamente equivalentes, ressaltando o ganho de robustez proveniente da arquitetura do Sistema Multiagente em si.

Para ilustrar estas conclusões de forma visual, a Figura 4.6 apresenta o Diagrama de Diferença Crítica (*Critical Difference Diagram*). Os algoritmos não conectados por barras horizontais possuem diferenças estatísticas significativas de desempenho, atestando a superioridade dos arranjos cooperativos.

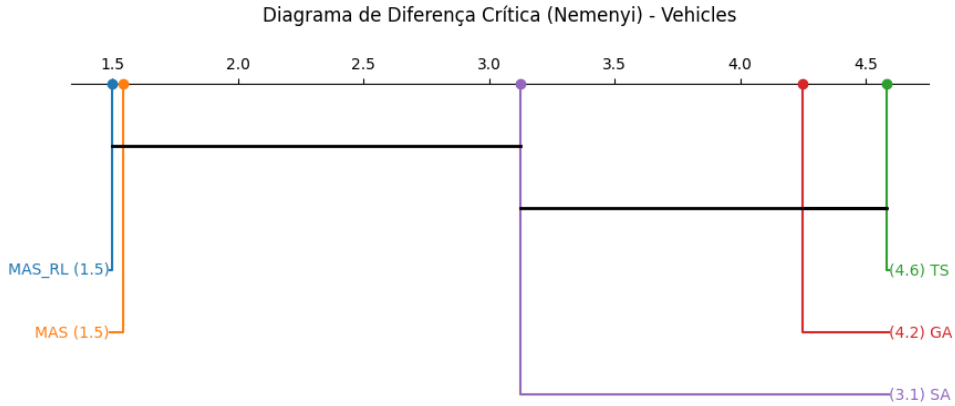


Figura 4.6: Diagrama de Diferença Crítica (Nemenyi) para a minimização de Veículos.

Esses resultados acarretam importantes implicações práticas para o modelo desenvolvido. A superioridade estatisticamente provada do *MAS* e do *MAS-RL* confirma que a arquitetura cooperativa mitiga o risco de paralisação em ótimos locais, gerando configurações de frota consideravelmente melhores nas instâncias de Solomon sem que esse ganho seja decorrente do acaso. Em relação ao *SA*, embora a diferença

de desempenho seja evidente, a ausência de significância estatística estrita sugere que, para algumas instâncias, o SA pode alcançar resultados próximos aos sistemas multiagente, reforçando a necessidade de análises mais granulares por cenário.

Capítulo 5

Conclusão

Este capítulo sintetiza as contribuições do presente trabalho, retomando os objetivos iniciais e apresentando as principais conclusões derivadas dos experimentos computacionais. Discute-se, ainda, as limitações observadas durante o desenvolvimento e as direções futuras para o aprimoramento do sistema multiagente proposto.

5.1 Resumo dos resultados finais

A investigação realizada demonstrou que a arquitetura multiagente e sua variante com aprendizado por reforço representam uma alternativa robusta e superior às meta-heurísticas clássicas aplicadas isoladamente. Ao permitir a exploração cooperativa do espaço de busca, o sistema proposto foi capaz de superar ou igualar o desempenho das linhas de base em todos os cenários de teste, confirmando a hipótese de que a inteligência coletiva atenua a sensibilidade das meta-heurísticas a configurações iniciais e topologias instáveis.

Os resultados agregados, corroborados pelas análises individuais por instância e, principalmente, o teste de hipóteses, evidenciam que a cooperação entre agentes especialistas permite uma otimização mais rápida e estável. Em relação aos métodos tradicionais, como *SA*, *GA* e *TS*, a estrutura de agência proposta convergiu de maneira **mais consistente, mais rápida e mais eficaz**, validando a eficácia da abordagem híbrida no tratamento do *VRPTW*.

Volta-se a ressaltar que, embora o sistema multiagente tenha demonstrado superioridade, a implementação atual do *Q-Learning* não apresentou ganhos significativos em relação à versão sem aprendizado. Esta limitação na eficácia do agente de reforço pode ser atribuída a restrições impostas pelo ambiente experimental. Primeiramente, o número de iterações e *steps* de aprendizado pode ter sido insuficiente para que o modelo convergisse para políticas de decisão otimizadas. Além disso, a busca por um equilíbrio entre o tempo total de processamento e a profundidade de exploração criou um dilema computacional: o incremento de *steps* para permitir um aprendizado mais robusto inevitavelmente reduz o tempo disponível para a busca local dos agentes, prejudicando a qualidade da solução final. Tais resultados indicam que, sob recursos computacionais e de tempo limitados, a complexidade adicional do *MAS RL* pode não ser totalmente compensada, sugerindo um campo fértil para futuras investigações em políticas de aprendizado mais eficientes.

5.2 Oportunidades de melhoria

No âmbito do aprimoramento do trabalho em si, identificou-se a necessidade de evoluir a modelagem do estado no *Q-Learning*. Uma proposta promissora, além de evidentemente aumentar o número de *steps* e rever os parâmetros de aprendizado, é a implementação de herança de estados, permitindo que parte do valor-*Q* de um estado anterior seja transmitido para o sucessor, acelerando a convergência do sistema. É possível, também, iniciar uma execução de otimização com uma tabela-*Q* já existente (*warm start*) de um treinamento anterior, o que aceleraria a convergência e os resultados. Adicionalmente, a otimização dos parâmetros das meta-heurísticas pode ser realizada através de abordagens teóricas ou empíricas mais refinadas.

Quanto à expansão do trabalho e adições estruturais, sugere-se a hibridização profunda através de Algoritmos Meméticos, utilizando o conhecimento adquirido sobre o desempenho individual das meta-heurísticas para compensar fraquezas específicas de cada uma, como já realizado em parte pelo *MAS*. Para aumentar a validade estatística do modelo, é fundamental testar o sistema em mais instâncias de dados com diferentes distribuições geográficas e números de pontos, evitando o viés de sobreajuste e aumentando o número de execuções por instância. Por fim, a aplicação em

casos reais [17], incorporando mapas georreferenciados e tempos de deslocamento não lineares, permitiria avaliar a robustez do *MAS-RL* frente às imprevisibilidades do transporte urbano, aproximando a pesquisa acadêmica das demandas práticas do setor de logística.

Outro ponto para o aumento da verossimilhança é o ajuste da função de custo para considerar explicitamente o tempo de espera, já que este fator representou uma parcela significativa do tempo total da rota nos experimentos realizados, impactando diretamente na eficiência logística.

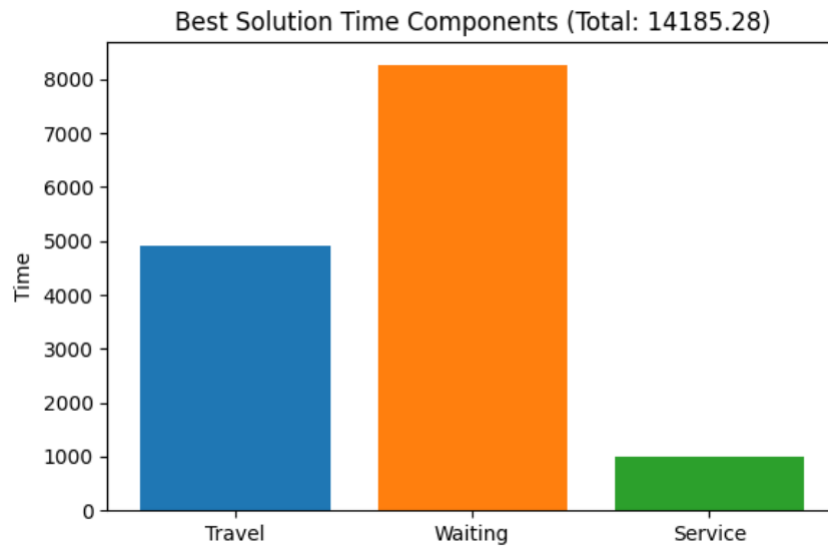


Figura 5.1: Tempos gastos com viagem, espera e serviço em uma execução simples de otimização para a instância *R201*.

Nota-se, conforme ilustrado na Figura 5.1, uma desproporção crítica entre o tempo de deslocamento e a ociosidade por espera, evidenciando ineficiências que impactam a viabilidade real das rotas. Visto que o tempo de serviço total é invariante e fixo, a elevada espera indica chegadas prematuras às janelas de tempo. Esse cenário reforça o desafio de sincronização temporal do *VRPTW*, validando a necessidade de mecanismos de controle para mitigar a inatividade operacional.

Referências Bibliográficas

- [1] GAREY, M. R., JOHNSON, D. S., *Computers and intractability: a guide to the theory of NP-completeness*. San Francisco, W. H. Freeman & Co., 1979.
- [2] DANTZIG, G. B., RAMSER, J. H., “The truck dispatching problem”, *Management Science*, v. 6, n. 1, pp. 80–91, 1959.
- [3] SOLOMON, M. M., “Algorithms for the vehicle routing and scheduling problems with time window constraints”, *Operations Research*, v. 35, n. 2, pp. 254–265, 1987.
- [4] VIDAL, T., LAPORTE, G., MATL, P., “A concise guide to existing and emerging vehicle routing problem variants”, *European Journal of Operational Research*, v. 286, n. 2, pp. 401–416, 2020.
- [5] KIRKPATRICK, S., GELATT, C. D., VECCHI, M. P., “Optimization by simulated annealing”, *Science*, v. 220, n. 4598, pp. 671–680, 1983.
- [6] GLOVER, F., “Tabu search: part I”, *ORSA Journal on Computing*, v. 1, n. 3, pp. 190–206, 1989.
- [7] HOLLAND, J. H., *Adaptation in natural and artificial systems: an introductory analysis with applications to biology, control, and artificial intelligence*. Ann Arbor, University of Michigan Press, 1975.
- [8] GOLDBERG, D. E., *Genetic algorithms in search, optimization, and machine learning*. Reading, Addison-Wesley, 1989.
- [9] WOOLDRIDGE, M., JENNINGS, N. R., “Intelligent agents: theory and practice”, *The Knowledge Engineering Review*, v. 10, n. 2, pp. 115–152, 1995.

- [10] Ter Hoeven, E., OTHERS, “Mesa 3: agent-based modeling with Python in 2025”, *Journal of Open Source Software*, v. 10, n. 107, pp. 7668, 2025.
- [11] GLOVER, F., LAGUNA, M., MARTÍ, R., “Fundamentals of scatter search and path relinking”, *Control and Cybernetics*, v. 29, n. 3, pp. 653–684, 2000.
- [12] SUTTON, R. S., BARTO, A. G., *Reinforcement learning: an introduction*. 2 ed. Cambridge, MIT Press, 2018.
- [13] WATKINS, C. J. C. H., DAYAN, P., “Q-learning”, *Machine Learning*, v. 8, n. 3-4, pp. 279–292, 1992.
- [14] SINTEF, “Vehicle Routing Problem with Time Windows: 100 Customers”, <https://www.sintef.no/projectweb/top/vrptw/100-customers/>, (Acesso em 06 de março de 2026).
- [15] MESTER, D., BRÄYSY, O., DULLAERT, W., *A Multi-parametric Evolution Strategies Algorithm for Vehicle Routing Problems*, Working paper, Institute of Evolution, University of Haifa, Israel, 2005.
- [16] BURKE, E. K., OTHERS, “Hyper-heuristics: a survey of the state of the art”, *Journal of the Operational Research Society*, v. 64, n. 12, pp. 1695–1724, 2013.
- [17] NVlabs, “olist-vrp-benchmark: Problem instances for the Vehicle Routing Problem”, <https://github.com/NVlabs/olist-vrp-benchmark>, (Acesso em 06 de março de 2026).

Apêndice A

Benchmark de Solomon

No contexto de otimização e logística, as instâncias de *Solomon* constituem um conjunto de problemas de teste (*benchmarks*) propostos por Marius Solomon em 1987 para o VRPTW. Essas instâncias são fundamentais para aferir a eficiência de algoritmos, pois representam diferentes desafios geográficos e temporais. As nomenclaturas **C**, **R** e **RC** referem-se à disposição espacial dos clientes:

- **Tipo C (*Clustered*)**: Os clientes estão organizados em agrupamentos (*clusters*). O algoritmo deve ser eficiente em atender aos clientes de um grupo denso antes de se deslocar para o próximo. Geralmente, este perfil é considerado o de mais fácil resolução para a obtenção de rotas ótimas.

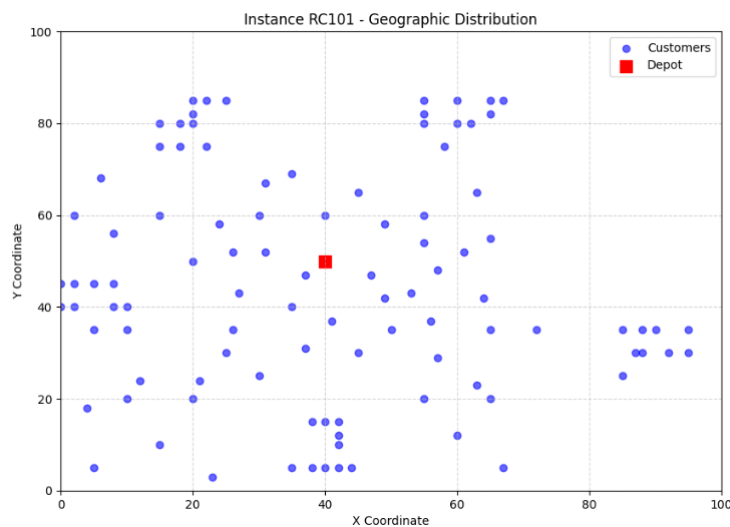


Figura A.1: Distribuição geográfica dos clientes na instância *Clustered C101*

- **Tipo R (*Random*)**: Os clientes estão distribuídos de forma aleatória e uni-

forme no plano de coordenadas. A ausência de padrões óbvios exige que o algoritmo avalie criteriosamente as distâncias e as janelas de tempo para cada tomada de decisão.

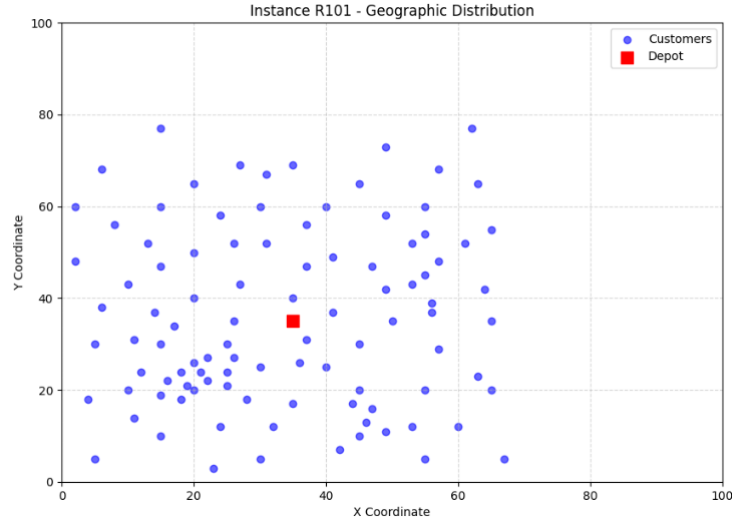


Figura A.2: Distribuição geográfica dos clientes na instância *Random* R101

- **Tipo RC (*Random-Clustered*):** Trata-se de um modelo híbrido que combina regiões de distribuição aleatória com agrupamentos definidos. Este cenário aproxima-se da realidade urbana complexa, onde centros comerciais densos convivem com áreas de entregas residenciais esparsas.

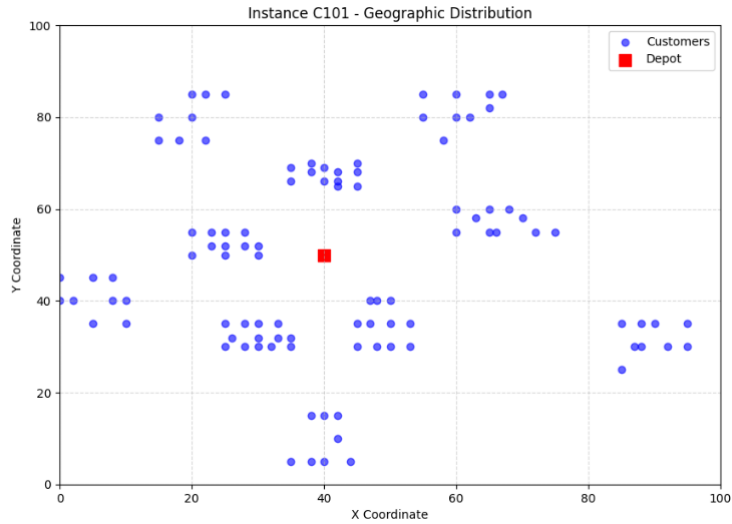


Figura A.3: Distribuição geográfica dos clientes na instância *Random-Clustered* RC101

Adicionalmente, as séries **1** e **2** definem o rigor das restrições operacionais, como a capacidade de carga dos veículos e a amplitude das janelas de tempo:

- **Série 1:** Apresenta restrições severas, com veículos de baixa capacidade de carga e janelas de tempo estreitas. Tais limitações resultam em rotas curtas e exigem uma frota maior para cobrir todos os pontos. Como exemplo, a instância *RC101* possui uma janela de tempo média de apenas 32,08.
- **Série 2:** Oferece um cenário mais flexível, com veículos de grande capacidade e janelas de tempo expandidas. Um único veículo consegue atender a uma quantidade maior de clientes, transferindo o desafio principal para a minimização da distância total percorrida. A instância *RC201*, utilizada nos testes deste trabalho, possui uma janela média significativamente superior, de 128,32.

Apêndice B

Estimativa para busca exaustiva

Este apêndice dedica-se a estimar o tempo necessário para a obtenção de uma solução ótima, por força bruta, referente à instância RC201, a qual contém 100 clientes (incluindo o depósito). O objetivo fundamental desta seção é evidenciar a natureza combinatória da complexidade temporal intrínseca a problemas de otimização de rotas *NP-hard* [1], como o *VRP*. Mesmo adotando a simplificação extrema do cenário para uma formulação análoga ao clássico Problema do Caixeiro Viajante (*TSP*), onde um único veículo visita sequencialmente todos os destinos, as permutações resultantes exigem um esforço computacional assintoticamente proibitivo.

O número de sequências possíveis para um veículo visitar todos os pontos e retornar à origem é dado por:

$$99! \approx 9,3326 \times 10^{155}$$

Para este cenário, assumimos um supercomputador teórico capaz de avaliar 1 bilhão (10^9) de soluções por milissegundo. Calculando a vazão anual desta máquina:

$$10^{12} \times 60 \times 60 \times 24 \times 365,25 \approx 3,15 \times 10^{19} \text{ sol./ano}$$

O tempo estimado para encontrar o ótimo absoluto seria:

$$\text{Tempo} = \frac{9,33 \times 10^{155}}{3,15 \times 10^{19}} \approx 2,96 \times 10^{136} \text{ anos}$$

Para compreender a magnitude deste resultado, a idade estimada do universo é de aproximadamente $13,8 \times 10^9$ anos. Outra comparação é que há “apenas” 10^{80} átomos

no universo observável. Se tivéssemos começado o processamento no momento do *Big Bang*, até o presente momento teríamos percorrido uma fração tão pequena do espaço de busca que ela seria estatisticamente equivalente a 0%.

Apêndice C

Resultados dos experimentos

A tabela apresenta os resultados completos de cada método para todas as instâncias testadas, incluindo o número médio de veículos (K), a distância total média (D) e o tempo médio de execução. Os valores de K , D e tempo são apresentados com seus respectivos desvios padrão. O tempo é apresentado em segundos.

Tabela C.1: Resultados completos dos experimentos por instância e método.

Instância	Método	Ref. (K^*, D^*)	K Médio	D Médio	Tempo de CPU (s)
c101	SA	(10, 828)	18.2 ± 0.9	1564 ± 73	1458 ± 20
	TS	(10, 828)	18.8 ± 1.5	1542 ± 94	1559 ± 41
	GA	(10, 828)	19.1 ± 0.6	2802 ± 215	1616 ± 185
	MAS	(10, 828)	17.0 ± 0.7	1583 ± 115	3811 ± 512
	MAS-RL	(10, 828)	17.1 ± 0.7	1637 ± 165	3417 ± 165
c102	SA	(10, 828)	15.2 ± 0.9	1477 ± 53	1474 ± 170
	TS	(10, 828)	17.0 ± 1.1	1453 ± 59	1468 ± 172
	GA	(10, 828)	16.5 ± 0.7	2778 ± 230	1419 ± 13
	MAS	(10, 828)	15.0 ± 0.8	1476 ± 86	3270 ± 62
	MAS-RL	(10, 828)	14.8 ± 0.6	1530 ± 174	3144 ± 77

Continua na próxima página

Instância	Método	Ref. (K^*, D^*)	K Médio	D Médio	Tempo de CPU (s)
c201	SA	(3, 591)	11.1 ± 0.7	1439 ± 61	1273 ± 78
	TS	(3, 591)	12.1 ± 1.4	1423 ± 95	1212 ± 61
	GA	(3, 591)	11.3 ± 0.7	2635 ± 181	1267 ± 17
	MAS	(3, 591)	10.1 ± 0.9	1449 ± 90	2947 ± 97
	MAS-RL	(3, 591)	10.2 ± 1.0	1554 ± 126	2779 ± 110
c202	SA	(3, 591)	8.5 ± 0.5	1298 ± 55	1174 ± 128
	TS	(3, 591)	10.6 ± 1.4	1293 ± 76	1154 ± 37
	GA	(3, 591)	9.2 ± 0.6	2642 ± 203	1151 ± 11
	MAS	(3, 591)	7.6 ± 0.5	1306 ± 104	2681 ± 82
	MAS-RL	(3, 591)	7.7 ± 0.7	1295 ± 93	2516 ± 88
r101	SA	(19, 1650)	25.2 ± 0.8	1979 ± 36	1924 ± 168
	TS	(19, 1650)	25.5 ± 0.7	1927 ± 29	1870 ± 178
	GA	(19, 1650)	26.7 ± 0.8	2391 ± 91	1904 ± 199
	MAS	(19, 1650)	24.0 ± 1.1	1967 ± 76	4247 ± 101
	MAS-RL	(19, 1650)	24.0 ± 0.8	1967 ± 58	4008 ± 143
r102	SA	(17, 1486)	21.1 ± 1.1	1839 ± 57	1687 ± 31
	TS	(17, 1486)	21.6 ± 1.0	1744 ± 42	1669 ± 146
	GA	(17, 1486)	22.5 ± 1.0	2282 ± 102	1665 ± 19
	MAS	(17, 1486)	20.1 ± 0.6	1750 ± 31	3862 ± 117
	MAS-RL	(17, 1486)	20.5 ± 0.7	1753 ± 38	3593 ± 106
r201	SA	(4, 1252)	9.3 ± 0.8	1594 ± 24	1260 ± 137
	TS	(4, 1252)	12.5 ± 1.1	1482 ± 56	1255 ± 37
	GA	(4, 1252)	9.8 ± 0.8	2466 ± 148	1191 ± 13
	MAS	(4, 1252)	9.1 ± 0.6	1589 ± 76	3602 ± 294
	MAS-RL	(4, 1252)	9.0 ± 0.8	1580 ± 98	2667 ± 90
r202	SA	(3, 1191)	7.8 ± 0.8	1443 ± 33	1101 ± 17
	TS	(3, 1191)	11.1 ± 0.7	1430 ± 41	1156 ± 23
	GA	(3, 1191)	7.8 ± 0.4	2386 ± 136	1171 ± 148
	MAS	(3, 1191)	7.2 ± 0.4	1520 ± 197	2561 ± 66
	MAS-RL	(3, 1191)	7.4 ± 0.7	1490 ± 179	2453 ± 113

Continua na próxima página

Instância	Método	Ref. (K^*, D^*)	K Médio	D Médio	Tempo de CPU (s)
rc102	SA	(12, 1554)	19.0 ± 0.9	1952 ± 54	1644 ± 21
	TS	(12, 1554)	20.4 ± 0.7	1940 ± 45	1547 ± 17
	GA	(12, 1554)	21.4 ± 0.5	2587 ± 77	1626 ± 23
	MAS	(12, 1554)	18.1 ± 0.9	1961 ± 166	3638 ± 149
	MAS-RL	(12, 1554)	18.0 ± 0.9	1895 ± 76	3481 ± 117
rc103	SA	(11, 1261)	16.3 ± 0.5	1747 ± 55	1613 ± 172
	TS	(11, 1261)	17.7 ± 1.8	1743 ± 100	1518 ± 169
	GA	(11, 1261)	18.7 ± 0.7	2452 ± 92	1515 ± 14
	MAS	(11, 1261)	16.3 ± 1.1	1710 ± 110	3346 ± 64
	MAS-RL	(11, 1261)	16.2 ± 0.6	1666 ± 39	3281 ± 109
rc201	SA	(4, 1406)	10.5 ± 0.7	1853 ± 91	1221 ± 16
	TS	(4, 1406)	13.2 ± 1.0	1867 ± 85	1268 ± 44
	GA	(4, 1406)	10.5 ± 0.5	3183 ± 216	1299 ± 164
	MAS	(4, 1406)	9.5 ± 0.8	1956 ± 185	2762 ± 58
	MAS-RL	(4, 1406)	9.4 ± 1.0	1913 ± 74	2696 ± 72
rc204	SA	(3, 798)	5.2 ± 0.6	1262 ± 45	1078 ± 142
	TS	(3, 798)	5.8 ± 0.9	1307 ± 96	921 ± 24
	GA	(3, 798)	5.0 ± 0.0	2704 ± 215	936 ± 11
	MAS	(3, 798)	4.6 ± 0.5	1284 ± 69	2210 ± 49
	MAS-RL	(3, 798)	4.6 ± 0.5	1302 ± 129	2151 ± 61